Bharatiya Vidya Bhavan's

SARDAR PATEL INSTITUTE OF TECHNOLOGY

Department of Computer Engineering — Cryptography and Network Security Lab

Lab 9: Building and Securing a Two-Tier PKI with X.509

Field	Value
Name	Anusha Dalal
UID	2024300032
Date of run	29 October 2026
Environment	Ubuntu on WSL2 (Windows), nginx/1.24.0 (Ubuntu), curl/8.5.0, OpenSSL and tshark from the Ubuntu packages

Aim

To build a two-tier certificate authority (Root CA and Intermediate CA) with OpenSSL, issue server and client certificates, configure nginx for mutual TLS, revoke a certificate and enforce it with a CRL and an OCSP responder, analyse TLS 1.2 and TLS 1.3 handshakes, and reproduce the common certificate-validation failures.

Report layout

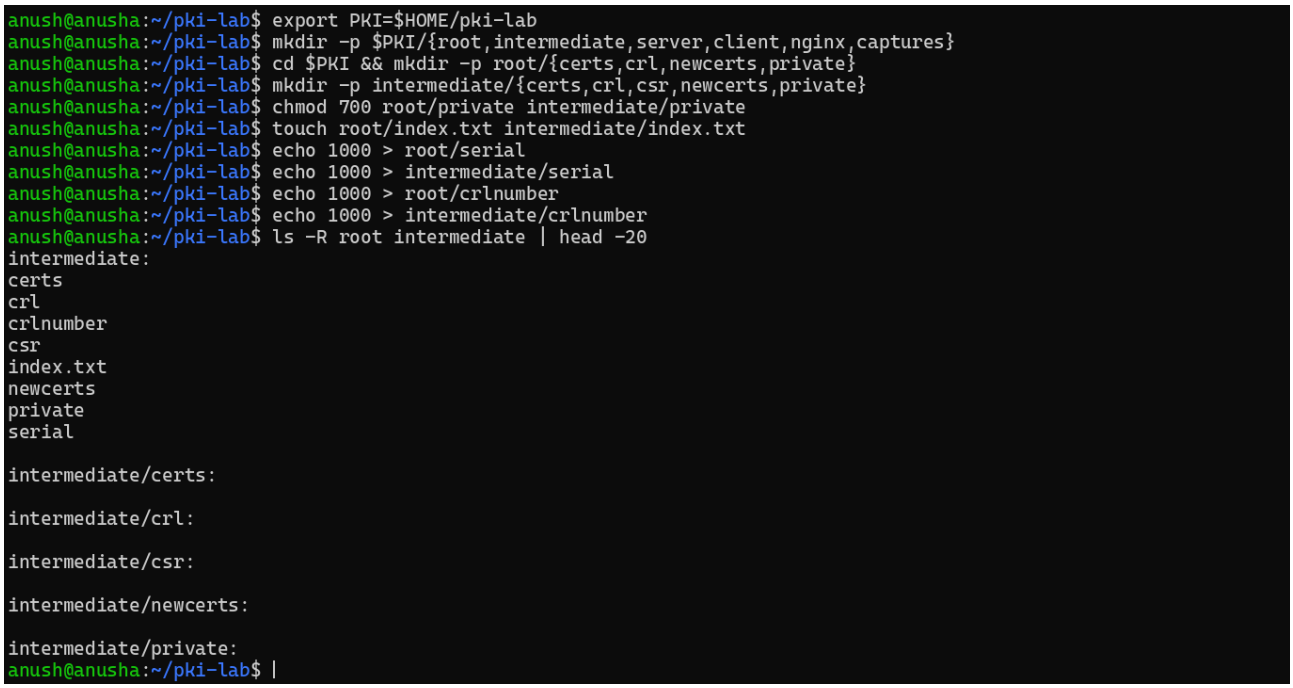
Section	Contents
Setup	Folder layout and CA databases
Part A	Root CA: configuration, key, self-signed certificate, inspection
Part B	Intermediate CA: CSR signed by the Root, chain file, pathlen and leaf-signing tests
Part C	Server and client certificates, verification, index.txt, purpose and SAN
Part D	Mutual TLS with nginx: valid client, missing client certificate, s_client
Part E	Revocation: CRLs, nginx ssl_crl, OCSP responder
Part F	TLS 1.2 and TLS 1.3 handshake captures (tshark output and Wireshark views)
Part G	Six failure scenarios with curl, OpenSSL and nginx error.log evidence
Reflection	Handwritten (150–250 words)
Submission	Files included in the zip (no private keys)

The answers to the Task questions follow each Part. The reflection is handwritten and attached at the marked place.

Setup

Working directory $PKI = ~/pki-lab with separate folders for the Root and the Intermediate CA. index.txt is each CA's database, serial the next serial number and crlnumber the next CRL number.

Figure 1: Setup: folder layout, index.txt, serial and crlnumber files



Part A — Root CA

root/openssl.cnf defines the Root CA (files, policy, default hash sha256) and the extension profiles v3_root_ca and v3_intermediate_ca. The Root key is 4096-bit RSA and the certificate is self-signed for 10 years (3650 days).

Figure 2: Part A.1: root/openssl.cnf

```
anush@anusha:~/pki-lab$ cat > root/openssl.cnf <<'EOF'
> [ ca ]
> default_ca = CA_default
>
> [ CA_default ]
> dir               = $ENV::PKI/root
> certs             = $dir/certs
> crl_dir           = $dir/crl
> new_certs_dir     = $dir/newcerts
> database          = $dir/index.txt
> serial            = $dir/serial
> crlnumber         = $dir/crlnumber
> private_key       = $dir/private/root.key.pem
> certificate       = $dir/certs/root.cert.pem
> crl               = $dir/crl/root.crl.pem
> default_md        = sha256
> default_days      = 365
> default_crl_days  = 30
> policy            = policy_loose
> unique_subject    = no
> copy_extensions   = none
>
> [ policy_loose ]
> countryName             = optional
> organizationName        = optional
> organizationalUnitName  = optional
> commonName              = supplied
>
> [ req ]
> default_md          = sha256
> distinguished_name  = req_dn
> prompt              = no
>
> [ req_dn ]
> CN = placeholder
>
> [ v3_root_ca ]
> subjectKeyIdentifier   = hash
> authorityKeyIdentifier = keyid:always,issuer
> basicConstraints       = critical, CA:true
> keyUsage               = critical, keyCertSign, cRLSign
>
> [ v3_intermediate_ca ]
> subjectKeyIdentifier   = hash
> authorityKeyIdentifier = keyid:always,issuer
> basicConstraints       = critical, CA:true, pathlen:0
> keyUsage               = critical, keyCertSign, cRLSign
>
> [ crl_ext ]
> authorityKeyIdentifier = keyid:always
> EOF
anush@anusha:~/pki-lab$
```

Figure 3: Part A.2: 4096-bit Root key and self-signed Root certificate (10 years)

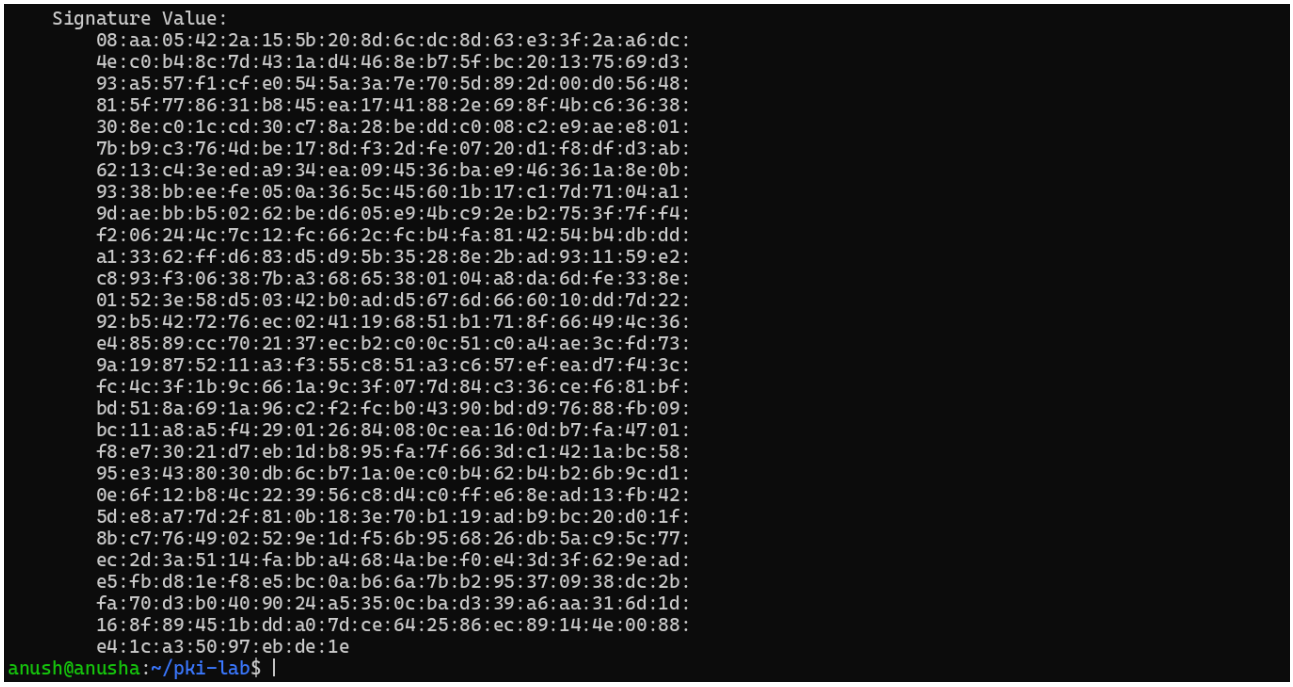
```
anush@anusha:~/pki-lab$ openssl genrsa -out root/private/root.key.pem 4096
anush@anusha:~/pki-lab$ chmod 400 root/private/root.key.pem
anush@anusha:~/pki-lab$ openssl req -config root/openssl.cnf -new -x509 -days 3650 -sha256 \
>   -extensions v3_root_ca \
>   -key root/private/root.key.pem \
>   -subj '/O=CSE Lab/CN=CSE Lab Root CA' \
>   -out root/certs/root.cert.pem
anush@anusha:~/pki-lab$ ls -l root/private root/certs
root/certs:
total 4
-rw-r--r-- 1 anush anush 1895 Oct 29 10:24 root.cert.pem

root/private:
total 4
-r-------- 1 anush anush 3268 Oct 29 10:24 root.key.pem
anush@anusha:~/pki-lab$
```

Figure 4: Part A.3: openssl x509 -text of the Root certificate (1/2)

```
anush@anusha:~/pki-lab$ openssl x509 -in root/certs/root.cert.pem -noout -text
Certificate:
    Data:
        Version: 3 (0x2)
        Serial Number:
            04:6d:bf:53:f3:58:1f:56:a2:96:8f:58:d6:bb:62:c4:a1:27:47:0a
        Signature Algorithm: sha256WithRSAEncryption
        Issuer: O = CSE Lab, CN = CSE Lab Root CA
        Validity
            Not Before: Oct 29 10:24:35 2026 GMT
            Not After : Oct 26 10:24:35 2036 GMT
        Subject: O = CSE Lab, CN = CSE Lab Root CA
        Subject Public Key Info:
            Public Key Algorithm: rsaEncryption
                Public-Key: (4096 bit)
                Modulus:
                    00:d8:71:87:58:98:a6:89:01:0d:09:c4:35:91:62:
                    cf:0f:ef:ba:22:50:a5:93:eb:6e:0f:a2:95:35:87:
                    9c:10:70:30:65:b9:a9:ad:ad:a7:c3:60:fd:a6:22:
                    a7:9c:dd:69:b0:4f:d0:e3:42:ba:07:db:cc:de:aa:
                    6a:69:b8:92:17:98:f1:e7:ee:63:b9:4c:63:4d:7e:
                    78:f1:59:8e:07:a1:e0:f2:71:95:ed:fc:f3:56:38:
                    45:ca:e7:8a:e9:49:31:45:33:b9:5f:1f:e7:e4:c7:
                    04:35:22:0a:04:b5:1a:dc:e9:f6:f6:b7:bb:3d:ec:
                    09:7e:ae:59:f1:6c:87:6c:c3:d3:af:d5:bb:b7:15:
                    94:6f:6e:e9:4d:a3:a0:0a:fe:9f:72:25:6b:7d:8a:
                    67:90:b4:d3:62:0b:63:5f:f8:fb:28:08:74:e5:3d:
                    45:97:3d:dc:c4:00:93:b9:93:1b:a0:d0:27:48:a4:
                    8b:3e:ad:7d:8a:87:68:15:ba:48:eb:b9:37:43:4f:
                    58:d3:15:78:76:59:61:d5:5e:24:ca:ef:8c:34:53:
                    a2:74:20:86:24:a1:be:1d:72:e5:f2:94:af:56:d6:
                    dd:92:23:69:e3:e4:c0:a8:5b:7f:be:b0:a0:c1:c8:
                    42:9f:6f:18:04:80:0f:80:c9:b2:58:13:aa:b7:b6:
                    85:c6:c6:b1:7d:5e:37:96:a7:8a:37:94:51:3f:44:
                    42:11:6c:a8:da:19:f2:ec:59:14:45:24:a3:f9:5a:
                    dd:d2:d7:4f:3f:47:a8:ed:b6:b0:2b:c5:b9:a3:0b:
                    28:c3:42:37:dd:53:59:58:b0:ee:eb:7e:9a:e5:95:
                    14:d8:bf:25:00:10:0e:ae:d8:da:00:dc:33:a9:7f:
                    34:46:5b:1c:1b:5b:86:fc:5c:4b:72:25:8a:7a:5a:
                    aa:22:83:fe:da:55:e8:94:bb:26:b2:b8:54:a0:50:
                    f9:fc:bc:ae:d1:08:6c:4a:ae:36:a3:c6:f0:14:da:
                    09:88:df:5e:e1:6a:43:f2:a6:cf:7b:73:c0:4d:49:
                    a9:31:d4:4d:af:ed:80:ac:1f:70:71:43:3e:2b:e7:
                    ca:65:90:8d:04:66:23:5a:ed:b5:d2:a8:26:c1:46:
                    ba:ad:c9:40:d6:b3:5d:bd:33:b6:28:30:8a:ba:89:
                    ab:e3:b5:e8:41:45:d8:4f:69:c4:eb:e7:71:7c:ff:
                    e5:8e:d9:ea:9e:ab:4f:e8:7a:4e:8d:15:45:14:45:
                    c2:66:23:7b:d9:f3:88:68:6d:75:a7:b4:81:49:60:
                    a2:aa:55:ce:6a:28:9d:a7:74:be:97:29:9f:ea:88:
                    c3:8d:c8:b6:6f:d4:6a:bf:63:c4:a5:ad:5f:ec:27:
                    7d:71:ef
                Exponent: 65537 (0x10001)
        X509v3 extensions:
            X509v3 Subject Key Identifier:
                A6:97:F7:D3:A1:E4:FF:BA:EF:92:50:38:29:47:CD:55:5F:44:8A:5F
            X509v3 Authority Key Identifier:
                A6:97:F7:D3:A1:E4:FF:BA:EF:92:50:38:29:47:CD:55:5F:44:8A:5F
            X509v3 Basic Constraints: critical
                CA:TRUE
            X509v3 Key Usage: critical
                Certificate Sign, CRL Sign
    Signature Algorithm: sha256WithRSAEncryption
```

Figure 5: Part A.3: openssl x509 -text of the Root certificate (2/2)



Answers: Task A — Root CA

Recorded values

- Serial number: **04:6d:bf:53:f3:58:1f:56:a2:96:8f:58:d6:bb:62:c4:a1:27:47:0a** (a random 20-byte value; OpenSSL 3 makes it random for self-signed certificates)
- Validity: **10 years (3650 days)**: **Oct 29 10:24:35 2026 GMT** to **Oct 26 10:24:35 2036 GMT**
- Signature algorithm: **sha256WithRSAEncryption**
- Key size: **RSA 4096 bits**
- Extensions:
  - Subject Key Identifier (hash of the public key)
  - Authority Key Identifier (equals the SKI, because the root signs itself)
  - Basic Constraints: **critical, CA:TRUE**
  - Key Usage: **critical, Certificate Sign, CRL Sign**

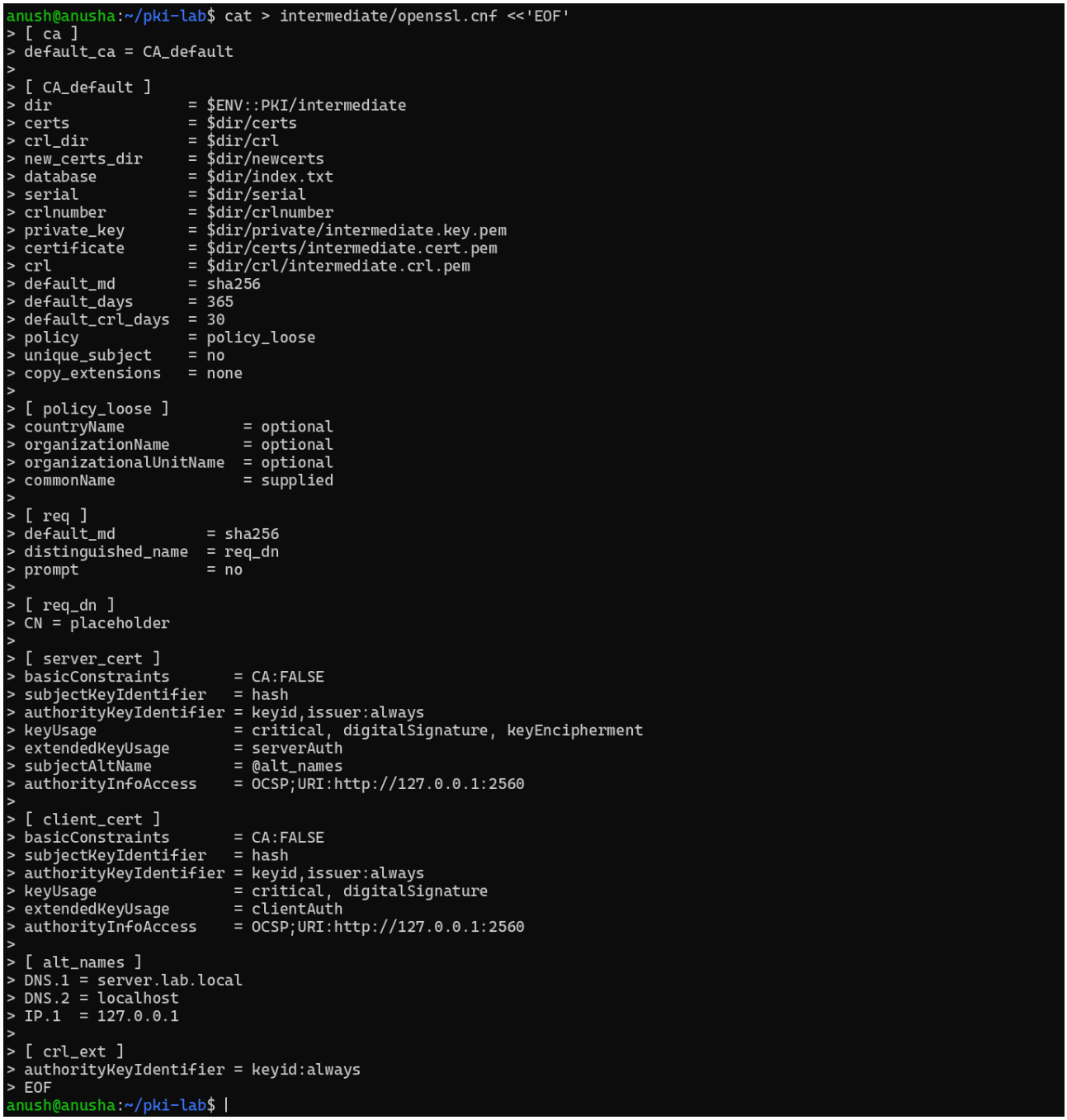
Why is basicConstraints critical? It decides whether a key may sign other certificates, which is the most security-sensitive fact in a certificate. "Critical" forces every program to obey it or reject the certificate, so no software can ignore it and treat a normal certificate as a CA (or the reverse). RFC 5280 requires it to be critical in CA certificates.

Why is a Root CA self-signed (issuer = subject)? It is the top of the chain, so nobody higher can sign it. Its signature only makes it a well-formed certificate; trust in it comes **out of band** (installed in a trust store, or passed with `--cacert`). That is also why its AKI equals its SKI.

Part B — Intermediate CA

The Intermediate creates its own 4096-bit key and a CSR, and the Root signs it with the v3_intermediate_ca profile (CA:TRUE, pathlen:0) for 5 years. intermediate/openssl.cnf also holds the server_cert and client_cert profiles, the SAN list and the OCSP URL.

Figure 6: Part B.1: intermediate/openssl.cnf



```
anush@anusha:~/pki-lab$ cat > intermediate/openssl.cnf <<'EOF'
> [ ca ]
> default_ca = CA_default
>
> [ CA_default ]
> dir               = $ENV::PKI/intermediate
> certs             = $dir/certs
> crl_dir           = $dir/crl
> new_certs_dir     = $dir/newcerts
> database          = $dir/index.txt
> serial            = $dir/serial
> crlnumber         = $dir/crlnumber
> private_key       = $dir/private/intermediate.key.pem
> certificate       = $dir/certs/intermediate.cert.pem
> crl               = $dir/crl/intermediate.crl.pem
> default_md        = sha256
> default_days      = 365
> default_crl_days  = 30
> policy            = policy_loose
> unique_subject    = no
> copy_extensions   = none
>
> [ policy_loose ]
> countryName             = optional
> organizationName        = optional
> organizationalUnitName  = optional
> commonName              = supplied
>
> [ req ]
> default_md          = sha256
> distinguished_name  = req_dn
> prompt              = no
>
> [ req_dn ]
> CN = placeholder
>
> [ server_cert ]
> basicConstraints       = CA:FALSE
> subjectKeyIdentifier   = hash
> authorityKeyIdentifier = keyid,issuer:always
> keyUsage               = critical, digitalSignature, keyEncipherment
> extendedKeyUsage       = serverAuth
> subjectAltName         = @alt_names
> authorityInfoAccess    = OCSP;URI:http://127.0.0.1:2560
>
> [ client_cert ]
> basicConstraints       = CA:FALSE
> subjectKeyIdentifier   = hash
> authorityKeyIdentifier = keyid,issuer:always
> keyUsage               = critical, digitalSignature
> extendedKeyUsage       = clientAuth
> authorityInfoAccess    = OCSP;URI:http://127.0.0.1:2560
>
> [ alt_names ]
> DNS.1 = server.lab.local
> DNS.2 = localhost
> IP.1  = 127.0.0.1
>
> [ crl_ext ]
> authorityKeyIdentifier = keyid:always
> EOF
anush@anusha:~/pki-lab$
```

Figure 7: Part B.2: Intermediate key and CSR, signed by the Root (openssl ca)

```
anush@anusha:~/pki-lab$ openssl genrsa -out intermediate/private/intermediate.key.pem 4096
anush@anusha:~/pki-lab$ chmod 400 intermediate/private/intermediate.key.pem
anush@anusha:~/pki-lab$ openssl req -config intermediate/openssl.cnf -new -sha256 \
>   -key intermediate/private/intermediate.key.pem \
>   -subj '/O=CSE Lab/CN=CSE Lab Intermediate CA' \
>   -out intermediate/csr/intermediate.csr.pem
anush@anusha:~/pki-lab$ openssl ca -batch -config root/openssl.cnf -extensions v3_intermediate_ca \
>   -days 1825 -notext -md sha256 \
>   -in intermediate/csr/intermediate.csr.pem \
>   -out intermediate/certs/intermediate.cert.pem
Using configuration from root/openssl.cnf
Check that the request matches the signature
Signature ok
The Subject's Distinguished Name is as follows
organizationName      :ASN.1 12:'CSE Lab'
commonName            :ASN.1 12:'CSE Lab Intermediate CA'
Certificate is to be certified until Oct 28 10:24:37 2031 GMT (1825 days)

Write out database with 1 new entries
Database updated
anush@anusha:~/pki-lab$
```

Figure 8: Part B.3: chain file and verification of the Intermediate against the Root

```
anush@anusha:~/pki-lab$ cat intermediate/certs/intermediate.cert.pem root/certs/root.cert.pem \
>   > intermediate/certs/ca-chain.cert.pem
anush@anusha:~/pki-lab$ openssl verify -CAfile root/certs/root.cert.pem intermediate/certs/intermediate.cert.pem
intermediate/certs/intermediate.cert.pem: OK
anush@anusha:~/pki-lab$ cat root/index.txt
V       3110281024377           1000    unknown /O=CSE Lab/CN=CSE Lab Intermediate CA
anush@anusha:~/pki-lab$
```

Figure 9: Part B: openssl x509 -text of the Intermediate certificate (1/2)

```
anush@anusha:~/pki-lab$ openssl x509 -in intermediate/certs/intermediate.cert.pem -noout -text
Certificate:
    Data:
        Version: 3 (0x2)
        Serial Number: 4096 (0x1000)
        Signature Algorithm: sha256WithRSAEncryption
        Issuer: O = CSE Lab, CN = CSE Lab Root CA
        Validity
            Not Before: Oct 29 10:24:37 2026 GMT
            Not After : Oct 28 10:24:37 2031 GMT
        Subject: O = CSE Lab, CN = CSE Lab Intermediate CA
        Subject Public Key Info:
            Public Key Algorithm: rsaEncryption
                Public-Key: (4096 bit)
                Modulus:
                    00:83:1b:e1:e6:cc:4e:08:21:30:db:69:3d:b3:08:
                    3c:25:2c:6b:fa:41:50:10:a4:5f:2f:4a:08:6e:d0:
                    2f:f5:bf:39:bc:a2:f7:f0:eb:99:d3:c3:7f:88:7c:
                    ee:05:dd:44:fb:09:d8:d3:73:b9:4a:8c:1b:73:64:
                    fe:c0:0e:ef:99:30:9b:49:f5:d4:71:36:5c:ba:01:
                    48:a3:2e:55:d8:76:e4:b9:43:66:17:26:ae:6b:2c:
                    29:dc:07:e2:d9:77:4d:f3:72:fb:0c:a9:3e:6d:ce:
                    86:e5:29:d4:f6:52:3f:5d:33:75:cf:f3:e8:b7:f8:
                    ee:e5:4c:b6:3d:57:18:09:08:b2:79:1c:bb:e4:de:
                    0a:e8:76:7d:13:06:1e:83:72:b9:89:6c:59:c4:49:
                    61:58:c3:8c:f1:0d:5b:05:01:10:7f:d0:ef:a4:7f:
                    73:3c:77:6c:3e:a4:c7:3f:48:ea:f2:d4:02:a1:d6:
                    79:04:73:6c:61:08:b4:b2:04:a3:f7:1e:b7:8d:ec:
                    af:df:26:bc:a7:7b:e8:ed:c2:36:e2:ae:b3:8d:0f:
                    24:86:4a:f7:c7:43:7f:34:70:20:28:c7:b9:8f:8f:
                    d8:3d:14:6a:af:70:bd:61:8a:5c:01:ac:d8:aa:53:
                    d7:97:1a:f0:74:de:11:5c:e2:d4:fa:92:57:ec:a0:
                    4a:6f:15:d2:96:2a:1f:5c:ff:6c:da:6d:32:84:d7:
                    03:49:8b:63:00:c9:d1:2b:8d:7e:90:0b:73:d9:44:
                    31:7a:ca:a5:d9:43:b0:1b:7e:d8:e4:e9:e8:fd:a3:
                    be:83:e9:fc:8c:32:64:3e:56:11:11:ac:15:49:a4:
                    ce:ec:3e:97:77:42:c8:f7:d7:5d:13:4a:48:9f:21:
                    b3:df:d0:6b:9e:76:09:e7:ff:2f:a0:d2:bb:c7:9a:
                    a2:cf:1b:3b:50:f2:1b:e7:25:c6:00:df:22:30:10:
                    a0:32:ae:71:26:f2:e7:ac:c0:96:58:a6:51:7a:c6:
                    8d:87:b8:dc:0b:f6:fc:c0:8d:46:8c:58:1f:18:b3:
                    0d:73:4a:8c:39:a7:16:e3:ec:0f:37:f5:39:17:29:
                    ca:dc:92:5c:96:84:cc:33:54:01:05:b0:f9:43:f0:
                    0e:9b:07:89:ce:f1:85:b4:66:69:51:e8:6e:27:a2:
                    8a:69:dc:2d:75:cd:db:de:41:61:ee:ae:1e:7b:b8:
                    53:d9:81:95:7d:5a:d6:6b:8f:df:fb:77:b0:0a:4c:
                    21:3a:21:72:c8:b5:a7:3e:0e:03:bb:33:91:f2:4e:
                    02:bd:e2:d3:60:e4:12:8f:d4:67:87:87:29:bc:09:
                    a9:da:b4:83:e4:d1:53:fc:f2:05:7a:c8:4f:26:a6:
                    21:c2:c1
                Exponent: 65537 (0x10001)
        X509v3 extensions:
            X509v3 Subject Key Identifier:
                BD:83:59:3B:51:56:9C:24:A6:A1:A8:6E:5B:4B:04:69:1B:9A:71:EA
            X509v3 Authority Key Identifier:
                A6:97:F7:D3:A1:E4:FF:BA:EF:92:50:38:29:47:CD:55:5F:44:8A:5F
            X509v3 Basic Constraints: critical
                CA:TRUE, pathlen:0
            X509v3 Key Usage: critical
                Certificate Sign, CRL Sign
    Signature Algorithm: sha256WithRSAEncryption
    Signature Value:
```

Figure 10: Part B: openssl x509 -text of the Intermediate certificate (2/2)

```
        78:a4:f4:b6:c1:4d:95:fa:07:d3:71:f8:1a:e9:5e:34:7c:0d:
        c8:ed:f3:fb:8a:2f:05:95:ac:fe:36:a2:d0:be:eb:cf:44:f4:
        aa:5a:fa:27:82:20:74:93:3c:e9:32:0a:1f:a4:73:61:f4:2c:
        0e:d8:dc:3d:22:ac:3c:ef:97:b2:9b:11:cb:1b:5e:19:84:b8:
        8b:df:44:56:9e:f4:e7:f1:84:4c:dd:66:04:5a:61:f7:6c:3b:
        c6:b6:dc:3a:2a:94:00:c4:dc:dd:75:37:1e:ef:3f:ab:06:5f:
        86:0d:94:ae:29:36:41:6c:4f:82:7e:4b:b6:52:89:21:3a:44:
        a0:5b:60:9f:d2:51:34:f4:d7:f7:de:89:0d:f1:dd:78:9d:19:
        2d:a6:e1:4b:4a:e1:17:51:7b:e9:1d:38:0e:ec:2b:f8:29:69:
        bf:76:6b:37:4f:e4:46:69:98:32:7e:c3:bb:46:de:df:22:b6:
        68:6b:ed:49:15:77:f1:62:e4:9d:3f:c9:70:95:3b:d2:ec:27:
        1c:1b:1e:e7:06:8e:02:b4:a5:86:6e:37:6c:5d:c1:63:5c:c6:
        91:67:73:ef:36:a4:03:72:4f:53:c2:8b:bb:f5:57:ea:dc:27:
        d8:0e:5a:fb:eb:a4:b4:61:68:98:18:23:36:98:13:00:4b:0e:
        f5:dc:c1:52:94:7a:9a:02:dd:3c:00:38:aa:e8:69:bd:e6:ac:
        ed:43:a1:1c:db:ee:d9:c6:52:55:6c:63:f2:2b:50:b8:34:b2:
        a3:15:cf:eb:65:8f:ca:bb:8f:ce:58:80:6f:81:9a:78:ac:70:
        ac:4a:34:ac:b4:87:06:19:f2:48:68:7b:04:a7:11:2a:b4:70:
        11:40:ca:7a:b1:dc:a5:2d:c4:5a:5d:ac:7f:d1:4f:b0:b4:28:
        bd:c8:0a:59:2c:27:65:7b:6d:ca:f1:ab:f0:65:b4:c3:fa:bf:
        43:5b:ba:f5:a5:ed:de:3a:ab:df:74:2b:bc:43:f4:b3:36:c7:
        21:81:e4:ae:fe:29:3c:7d:8e:54:52:d4:fb:98:15:a1:7d:a0:
        bd:fc:60:76:09:73:c2:fa:73:90:ec:38:9a:e2:6c:10:ad:20:
        86:87:a1:0a:01:28:f1:3d:20:d2:8e:b3:bc:74:5f:8f:2f:7a:
        98:a5:97:70:db:5e:45:32:66:6c:e4:0d:64:cc:1d:59:fe:0e:
        29:27:c6:f9:27:7e:9c:e4:05:36:eb:b8:bc:44:fc:f9:fe:4f:
        3d:33:a8:0a:25:c3:57:c4:6d:5a:69:9e:ab:bb:62:24:54:df:
        ee:26:42:70:bd:2f:5f:ce:8c:47:53:6f:22:20:6a:d0:e7:1e:
        10:7b:96:64:aa:2e:d0:8b
anush@anusha:~/pki-lab$
```

Figure 11: Task B: certificate signed by a leaf (Alice) and the pathlen:0 test

```
anush@anusha:~/pki-lab$ mkdir -p demo
anush@anusha:~/pki-lab$ openssl req -new -newkey rsa:2048 -nodes -keyout demo/evil.key.pem \
>   -subj '/O=Evil/CN=evil' -out demo/evil.csr.pem 2>/dev/null
anush@anusha:~/pki-lab$ openssl x509 -req -in demo/evil.csr.pem -CA client/alice.cert.pem -CAkey client/alice.key.pe
m \
>   -CAcreateserial -days 30 -out demo/evil.cert.pem
Certificate request self-signature ok
subject=O = Evil, CN = evil
anush@anusha:~/pki-lab$ openssl verify -CAfile intermediate/certs/ca-chain.cert.pem -untrusted client/alice.cert.pem
 demo/evil.cert.pem
O = CSE Lab, CN = alice
error 79 at 1 depth lookup: invalid CA certificate
O = CSE Lab, CN = CSE Lab Intermediate CA
error 25 at 2 depth lookup: path length constraint exceeded
O = CSE Lab, CN = alice
error 32 at 1 depth lookup: key usage does not include certificate signing
error demo/evil.cert.pem: verification failed
anush@anusha:~/pki-lab$ printf 'basicConstraints=critical,CA:true\nkeyUsage=critical,keyCertSign,cRLSign\n' > demo/s
ubca.ext
anush@anusha:~/pki-lab$ openssl req -new -newkey rsa:2048 -nodes -keyout demo/subca.key.pem \
>   -subj '/O=CSE Lab/CN=Sub CA' -out demo/subca.csr.pem 2>/dev/null
anush@anusha:~/pki-lab$ openssl x509 -req -in demo/subca.csr.pem -CA intermediate/certs/intermediate.cert.pem \
>   -CAkey intermediate/private/intermediate.key.pem -CAcreateserial -days 30 \
>   -extfile demo/subca.ext -out demo/subca.cert.pem
Certificate request self-signature ok
subject=O = CSE Lab, CN = Sub CA
anush@anusha:~/pki-lab$ openssl req -new -newkey rsa:2048 -nodes -keyout demo/leaf.key.pem \
>   -subj '/O=CSE Lab/CN=leaf' -out demo/leaf.csr.pem 2>/dev/null
anush@anusha:~/pki-lab$ openssl x509 -req -in demo/leaf.csr.pem -CA demo/subca.cert.pem -CAkey demo/subca.key.pem \
>   -CAcreateserial -days 30 -out demo/leaf.cert.pem
Certificate request self-signature ok
subject=O = CSE Lab, CN = leaf
anush@anusha:~/pki-lab$ cat demo/subca.cert.pem intermediate/certs/intermediate.cert.pem > demo/sub+intermediate.pem
anush@anusha:~/pki-lab$ openssl verify -CAfile root/certs/root.cert.pem -untrusted demo/sub+intermediate.pem demo/le
af.cert.pem
O = CSE Lab, CN = CSE Lab Intermediate CA
error 25 at 2 depth lookup: path length constraint exceeded
error demo/leaf.cert.pem: verification failed
anush@anusha:~/pki-lab$
```

Answers: Task B — Intermediate CA

Root vs Intermediate

	Root CA	Intermediate CA

Subject	O=CSE Lab, CN=CSE Lab Root CA	O=CSE Lab, CN=CSE Lab Intermediate CA
Issuer	itself	CSE Lab Root CA
Signed with	its own key	the Root's key
Validity	10 years	5 years (1825 days)
Key	RSA 4096	RSA 4096
Serial	random 20 bytes	0x1000 (from the Root's serial file)
Basic Constraints	critical, CA:TRUE	critical, CA:TRUE, **pathlen:0**
Key Usage	critical, keyCertSign, cRLSign	critical, keyCertSign, cRLSign
AKI	= its own SKI	= the **Root's** SKI

What does pathlen:0 do? It allows **zero** further CA certificates below the Intermediate, so it can sign only end-entity (leaf) certificates. If the Intermediate key is stolen, the thief still cannot create a valid sub-CA. My test: a Sub CA signed by the Intermediate, then a leaf signed by that Sub CA → **error 25: path length constraint exceeded**.

What if a certificate is issued from a leaf? It is rejected. A leaf has **CA:FALSE** and its key usage has no **keyCertSign**. My test (a certificate signed with Alice's key) → **error 79: invalid CA certificate** and **error 32: key usage does not include certificate signing**.

Part C — Server and client certificates

The Intermediate signs a 2048-bit server certificate (server_cert: serverAuth, SAN server.lab.local / localhost / 127.0.0.1, OCSP URL) and client certificates for Alice and Bob (client_cert: clientAuth). server.fullchain.pem = server certificate + Intermediate.

Figure 12: Part C.1: server key, CSR, certificate and full chain

```
anush@anusha:~/pki-lab$ openssl genrsa -out server/server.key.pem 2048
anush@anusha:~/pki-lab$ openssl req -new -key server/server.key.pem \
>   -subj '/O=CSE Lab/CN=server.lab.local' -out server/server.csr.pem
anush@anusha:~/pki-lab$ openssl ca -batch -config intermediate/openssl.cnf -extensions server_cert \
>   -days 365 -notext -md sha256 \
>   -in server/server.csr.pem -out server/server.cert.pem
Using configuration from intermediate/openssl.cnf
Check that the request matches the signature
Signature ok
The Subject's Distinguished Name is as follows
organizationName      :ASN.1 12:'CSE Lab'
commonName            :ASN.1 12:'server.lab.local'
Certificate is to be certified until Oct 29 10:24:38 2027 GMT (365 days)

Write out database with 1 new entries
Database updated
anush@anusha:~/pki-lab$ cat server/server.cert.pem intermediate/certs/intermediate.cert.pem > server/server.fullchai
n.pem
anush@anusha:~/pki-lab$
```

Figure 13: Part C.2: client certificates for Alice and Bob

```
anush@anusha:~/pki-lab$ for u in alice bob; do
>   openssl genrsa -out client/$u.key.pem 2048
>   openssl req -new -key client/$u.key.pem \
>     -subj "/O=CSE Lab/CN=$u" -out client/$u.csr.pem
>   openssl ca -batch -config intermediate/openssl.cnf -extensions client_cert \
>     -days 365 -notext -md sha256 \
>     -in client/$u.csr.pem -out client/$u.cert.pem
> done
Using configuration from intermediate/openssl.cnf
Check that the request matches the signature
Signature ok
The Subject's Distinguished Name is as follows
organizationName      :ASN.1 12:'CSE Lab'
commonName            :ASN.1 12:'alice'
Certificate is to be certified until Oct 29 10:24:38 2027 GMT (365 days)

Write out database with 1 new entries
Database updated
Using configuration from intermediate/openssl.cnf
Check that the request matches the signature
Signature ok
The Subject's Distinguished Name is as follows
organizationName      :ASN.1 12:'CSE Lab'
commonName            :ASN.1 12:'bob'
Certificate is to be certified until Oct 29 10:24:39 2027 GMT (365 days)

Write out database with 1 new entries
Database updated
anush@anusha:~/pki-lab$
```

Figure 14: Part C.3: verification of server and client certificates

```
anush@anusha:~/pki-lab$ openssl verify -CAfile intermediate/certs/ca-chain.cert.pem server/server.cert.pem
server/server.cert.pem: OK
anush@anusha:~/pki-lab$ openssl verify -CAfile intermediate/certs/ca-chain.cert.pem client/alice.cert.pem
client/alice.cert.pem: OK
anush@anusha:~/pki-lab$ openssl verify -CAfile intermediate/certs/ca-chain.cert.pem -purpose sslclient client/alice.
cert.pem
client/alice.cert.pem: OK
anush@anusha:~/pki-lab$
```

Figure 15: Task C: index.txt, Alice with -purpose sslserver, EKU and SAN

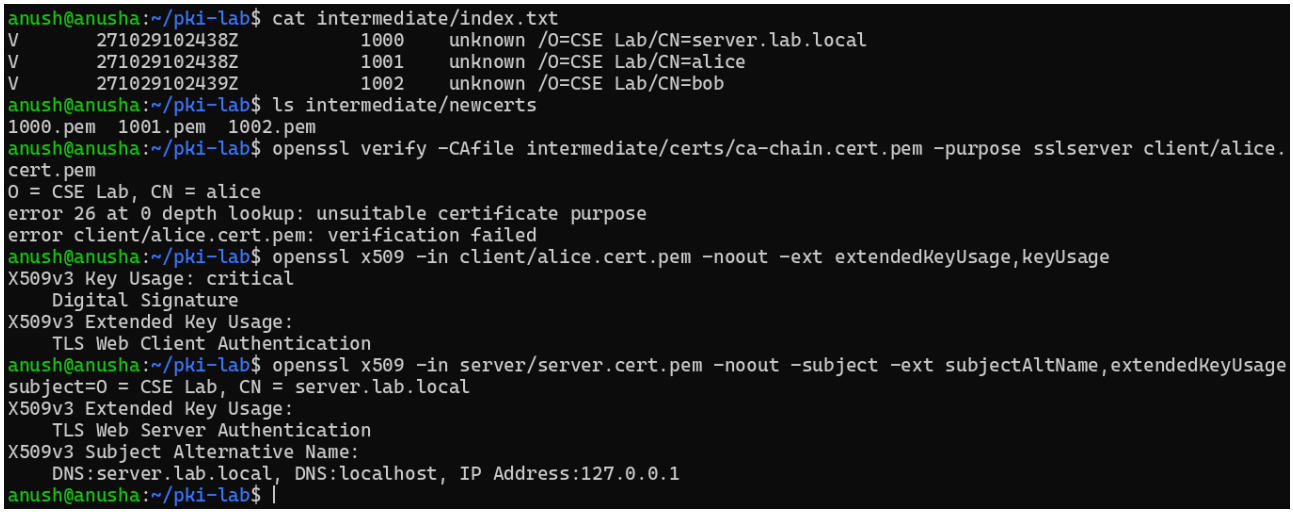
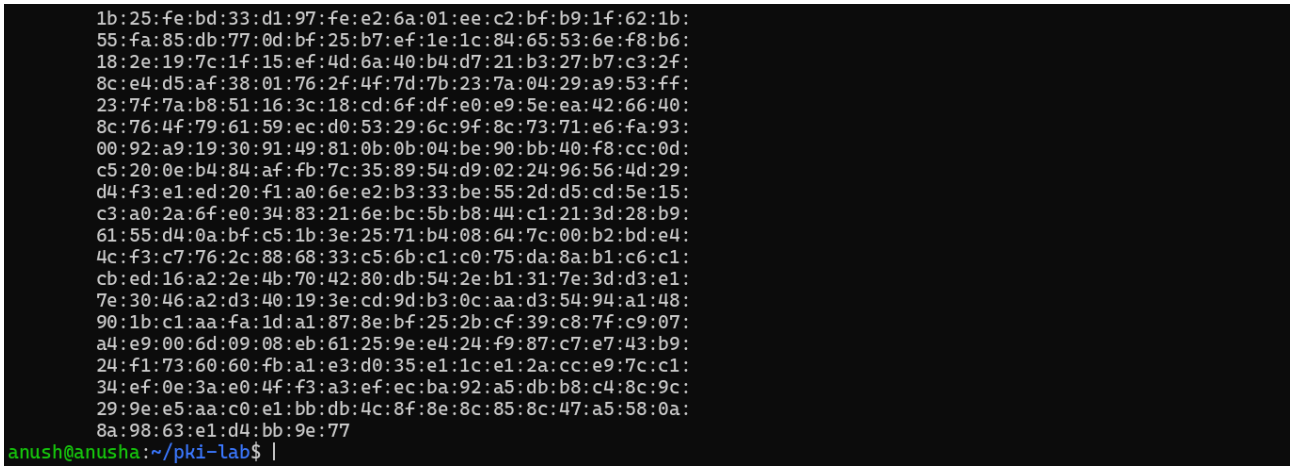


Figure 16: Part C: openssl x509 -text of the server certificate (1/2)

```
anush@anusha:~/pki-lab$ openssl x509 -in server/server.cert.pem -noout -text
Certificate:
    Data:
        Version: 3 (0x2)
        Serial Number: 4096 (0x1000)
        Signature Algorithm: sha256WithRSAEncryption
        Issuer: O = CSE Lab, CN = CSE Lab Intermediate CA
        Validity
            Not Before: Oct 29 10:24:38 2026 GMT
            Not After : Oct 29 10:24:38 2027 GMT
        Subject: O = CSE Lab, CN = server.lab.local
        Subject Public Key Info:
            Public Key Algorithm: rsaEncryption
                Public-Key: (2048 bit)
                Modulus:
                    00:be:dd:53:42:ea:44:2a:73:42:94:99:e0:1d:db:
                    d6:46:8b:5f:0c:f1:ce:25:78:3f:5d:3c:e1:9a:7c:
                    c3:26:c3:8c:96:c3:8d:75:fd:2b:b9:27:f5:ec:f9:
                    a5:fe:34:66:c9:9a:6d:12:29:59:cd:49:4d:63:e8:
                    34:b4:60:d2:f1:ef:8e:06:b4:a8:f7:49:d2:cb:ac:
                    63:22:0d:c2:e4:49:08:54:29:7c:cf:76:3a:d9:1b:
                    75:a5:25:d1:1e:5c:3a:91:c5:00:6e:bd:a4:e3:ed:
                    0b:9d:c7:ef:be:19:30:0e:1b:f9:b2:2b:5d:46:f2:
                    ea:d8:41:8e:5b:2a:1c:ae:7e:4d:5f:bc:74:3b:1b:
                    1f:83:93:dd:85:6f:d3:32:0a:6a:e4:a7:55:40:af:
                    5b:76:e6:42:d1:1b:58:21:0f:e0:5b:4b:2d:82:ef:
                    b6:0f:28:0f:37:4d:74:05:f8:45:f3:75:c4:a4:9d:
                    18:76:c3:17:d8:33:c6:ba:b1:2d:a2:52:61:3e:63:
                    e5:43:e8:46:47:c5:09:17:a2:b8:8d:d8:e0:cc:b5:
                    33:9b:34:9b:79:54:04:e7:d1:33:eb:48:89:0f:57:
                    4e:6f:fe:95:f3:24:6f:f7:0f:2e:73:e1:c3:b0:8b:
                    3e:cd:79:ad:04:9e:b8:65:34:dc:bc:25:5f:c8:26:
                    91:b9
                Exponent: 65537 (0x10001)
        X509v3 extensions:
            X509v3 Basic Constraints:
                CA:FALSE
            X509v3 Subject Key Identifier:
                8F:3C:FF:EB:ED:EA:3E:4A:D8:F9:0B:C2:D6:8F:4F:7C:2F:90:7D:0C
            X509v3 Authority Key Identifier:
                keyid:BD:83:59:3B:51:56:9C:24:A6:A1:A8:6E:5B:4B:04:69:1B:9A:71:EA
                DirName:/O=CSE Lab/CN=CSE Lab Root CA
                serial:10:00
            X509v3 Key Usage: critical
                Digital Signature, Key Encipherment
            X509v3 Extended Key Usage:
                TLS Web Server Authentication
            X509v3 Subject Alternative Name:
                DNS:server.lab.local, DNS:localhost, IP Address:127.0.0.1
            Authority Information Access:
                OCSP - URI:http://127.0.0.1:2560
    Signature Algorithm: sha256WithRSAEncryption
    Signature Value:
        50:eb:9b:f8:5a:8c:ae:84:ae:49:40:24:f4:71:3c:7a:86:e9:
        43:bd:04:e4:61:c5:90:95:e7:29:23:46:27:6d:45:52:a8:75:
        85:60:85:c2:a5:e7:74:a2:d7:77:a4:6a:98:d7:dc:83:c2:e3:
        c9:39:2e:eb:00:1e:10:b6:b6:34:66:4a:d3:86:93:2e:34:de:
        19:ea:f7:31:be:1e:1e:a3:f7:60:d0:bb:31:d8:14:02:8e:e4:
        e3:aa:15:ed:e7:30:7c:19:db:c9:4f:44:88:58:e6:8a:29:8f:
        c9:9a:a3:f2:2d:83:cd:e6:90:e2:98:b3:1d:b1:aa:d9:fa:78:
        3a:33:51:df:ba:fa:a9:a1:b4:82:44:42:86:30:20:3e:6c:31:
        bc:a1:16:d2:f9:9c:a4:14:f9:20:c6:fe:4e:78:17:1f:dc:a0:
```

Figure 17: Part C: openssl x509 -text of the server certificate (2/2)



Answers: Task C — Server and client certificates

1. index.txt columns (TAB-separated):

1. **Status:** V = valid, R = revoked, E = expired
2. **Expiry** date-time, YYMMDDHHMMSSZ (UTC)
3. **Revocation** date and reason (empty while valid)
4. **Serial** number in hex (1000 = server, 1001 = alice, 1002 = bob; same as the files in newcerts/)
5. File name (always "unknown")
6. **Subject** DN (e.g. /O=CSE Lab/CN=alice)

2. Alice with `-purpose sslserver`: fails with **error 26: unsuitable certificate purpose**. Cause: the **Extended Key Usage** extension says only *TLS Web Client Authentication* (clientAuth), so it is not valid as a server certificate.

3. Why SAN when CN has the hostname? The CN is an old free-text field. Modern clients (RFC 6125/2818, browsers since ~2017) check host names **only in the Subject Alternative Name**, a structured list that can hold several DNS names and IP addresses. Our SAN covers server.lab.local, localhost and 127.0.0.1, so one certificate works for all three.

Part D — Mutual TLS with nginx

nginx listens on 8443 with the server full chain, requires a client certificate (ssl_verify_client on) and verifies it against the CA chain (ssl_client_certificate, ssl_verify_depth 2). The response prints the client DN, the verification result and the TLS version.

Figure 18: Part D.1: nginx.conf and starting nginx on port 8443

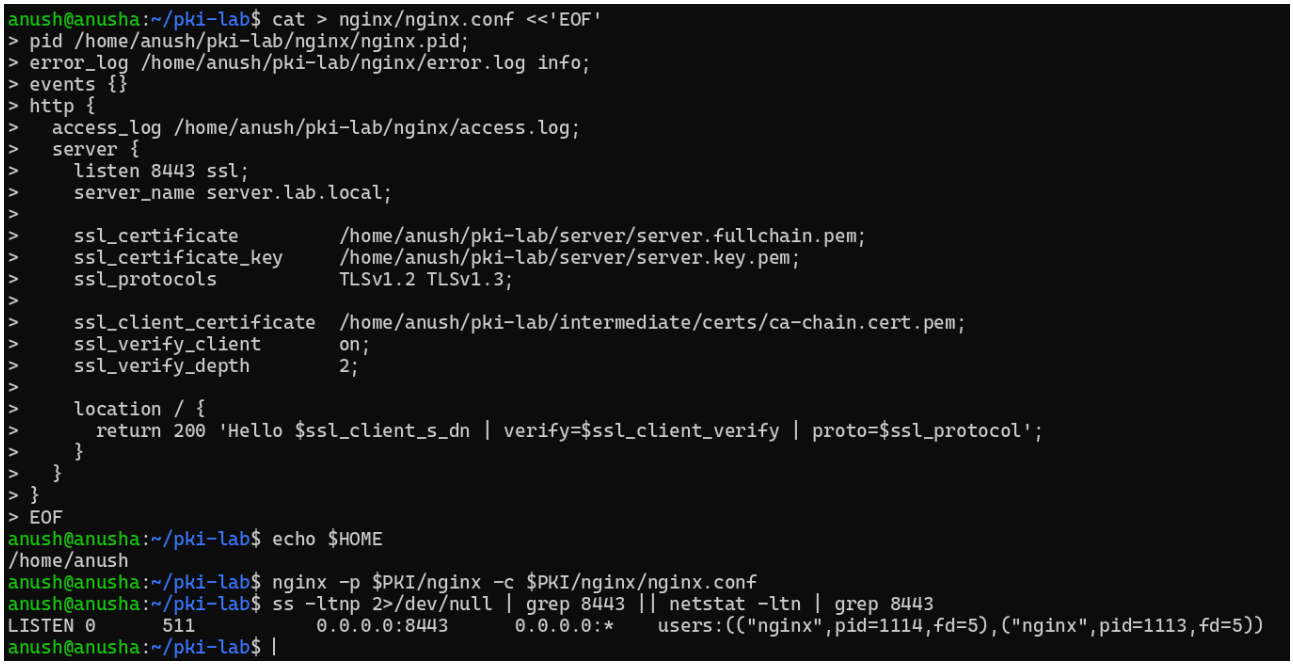


Figure 19: Part D.2: valid client (Alice) and no client certificate (HTTP 400), with error.log

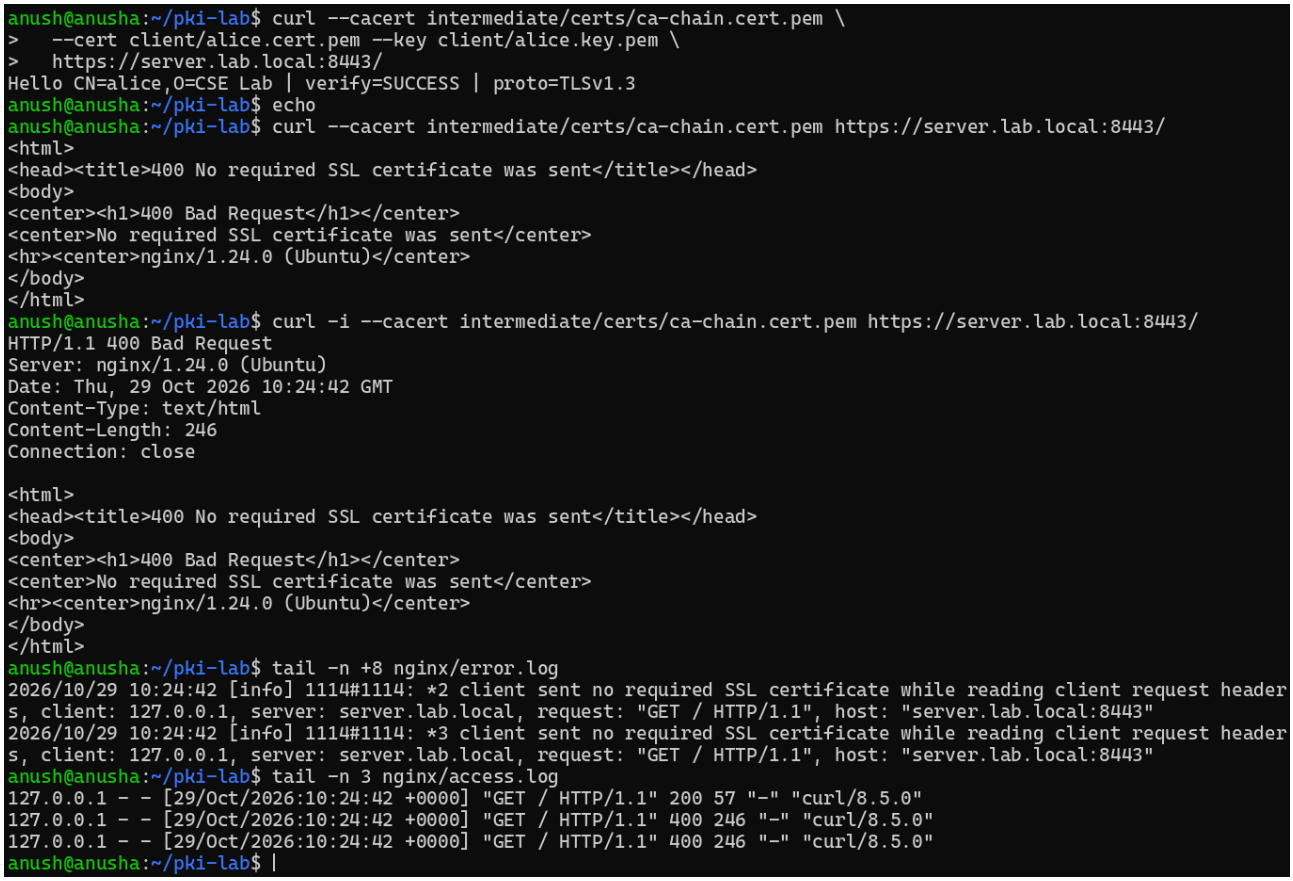
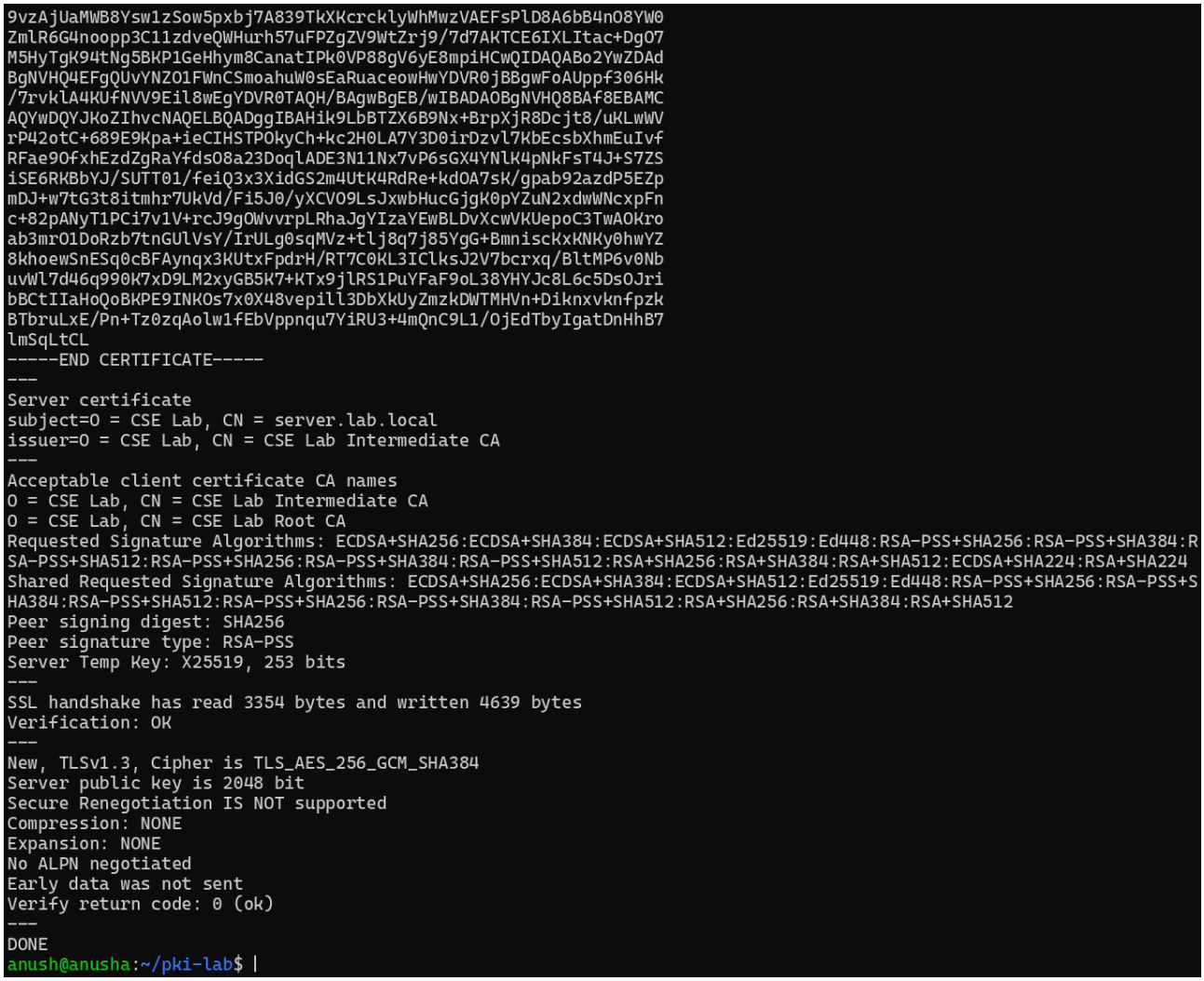


Figure 20: Part D.2: openssl s_client -showcerts (chain and acceptable client CA names) (1/2)

```
anush@anusha:~/pki-lab$ openssl s_client -connect server.lab.local:8443 -CAfile intermediate/certs/ca-chain.cert.pem
 \
>   -cert client/alice.cert.pem -key client/alice.key.pem -showcerts < /dev/null
CONNECTED(00000003)
depth=2 O = CSE Lab, CN = CSE Lab Root CA
verify return:1
depth=1 O = CSE Lab, CN = CSE Lab Intermediate CA
verify return:1
depth=0 O = CSE Lab, CN = server.lab.local
verify return:1
---
Certificate chain
 0 s:O = CSE Lab, CN = server.lab.local
   i:O = CSE Lab, CN = CSE Lab Intermediate CA
   a:PKEY: rsaEncryption, 2048 (bit); sigalg: RSA-SHA256
   v:NotBefore: Oct 29 10:24:38 2026 GMT; NotAfter: Oct 29 10:24:38 2027 GMT
-----BEGIN CERTIFICATE-----
MIIE6jCCAtKgAwIBAgICEAAwDQYJKoZIhvcNAQELBQAwNDEQMA4GA1UECgwHQ1NF
IExhYjEgMB4GA1UEAwwXQ1NFIExhYiBJbnRlcm1lZGlhdGUgQ0EwHhcNMjYxMDI5
MTAyNDM4WhcNMjcxMDI5MTAyNDM4WjAtMRAwDgYDVQQKDAdDU0UgTGFiMRkwFwYD
VQQDDBBzZXJ2ZXIubGFiLmxvY2FsMIIBIjANBgkqhkiG9w0BAQEFAAOCAQ8AMIIB
CgKCAQEAvt1TQupEKnNClJngHdvWRotfDPHOJXg/XTzhmnzDJsOMlsONdf0ruSf1
7Pml/jRmyZptEilZzUlNY+g0tGDS8e+OBrSo90nSy6xjIg3C5EkIVCl8z3Y62Rt1
pSXRHlw6kcUAbr2k4+0LncfvvhkwDhv5sitdRvLq2EGOWyocrn5NX7x0Oxsfg5Pd
hW/TMgpq5KdVQK9bduZC0RtYIQ/gW0stgu+2DygPN010BfhF83XEpJ0YdsMX2DPG
urEtolJhPmPlQ+hGR8UJF6K4jdjgzLUzmzSbeVQE59Ez60iJD1dOb/6V8yRv9w8u
c+HDsIs+zXmtBJ64ZTTcvCVfyCaRuQIDAQABo4IBCzCCAQcwCQYDVR0TBAIwADAd
BgNVHQ4EFgQUjzz/6+3qPkrY+QvC1o9PfC+QfQwwVQYDVR0jBE4wTIAUvYNZO1FW
nCSmoahuW0sEaRuaceqhMKQuMCwxEDAOBgNVBAoMB0NTRSBMYWIxGDAWBgNVBAMM
D0NTRSBMYWIgUm9vdCBDQYICEAAwDgYDVR0PAQH/BAQDAgWgMBMGA1UdJQQMMAoG
CCsGAQUFBwMBMCwGA1UdEQQlMCOCEHNlcnZlci5sYWIubG9jYWyCCWxvY2FsaG9z
dIcEfwAAATAxBggrBgEFBQcBAQQlMCMwIQYIKwYBBQUHMAGGFWh0dHA6Ly8xMjcu
MC4wLjE6MjU2MDANBgkqhkiG9w0BAQsFAAOCAgEAUOub+FqMroSuSUAk9HE8eobp
Q70E5GHFkJXnKSNGJ21FUqh1hWCFwqXndKLXd6RqmNfcg8LjyTku6wAeELa2NGZK
04aTLjTeGer3Mb4eHqP3YNC7MdgUAo7k46oV7ecwfBnbyU9EiFjmiimPyZqj8i2D
zeaQ4pizHbGq2fp4OjNR37r6qaG0gkRChjAgPmwxvKEW0vmcpBT5IMb+TngXH9yg
GyX+vTPRl/7iagHuwr+5H2IbVfqF23cNvyW37x4chGVTbvi2GC4ZfB8V701qQLTX
IbMnt8MvjOTVrzgBdi9PfXsjegQpqVP/I396uFEWPBjNb9/g6V7qQmZAjHZPeWFZ
7NBTKWyfjHNx5vqTAJKpGTCRSYELCwS+kLtA+MwNxSAOtISv+3w1iVTZAiSWVk0p
1PPh7SDxoG7iszO+VS3VzV4Vw6Aqb+A0gyFuvFu4RMEhPSi5YVXUCr/FGz4lcbQI
ZHwAsr3kTPPHdiyIaDPFa8HAddqKscbBy+0Woi5LcEKA21QusTF+PdPhfjBGotNA
GT7NnbMMqtNUlKFIkBvBqvodoYeOvyUrzznIf8kHpOkAbQkI62ElnuQk+YfH50O5
JPFzYGD7oePQNeEc4SrM6XzBNO8OOuBP86Pv7LqSpdu4xIycKZ7lqsDhu9tMj46M
hYxHpVgKiphj4dS7nnc=
-----END CERTIFICATE-----
 1 s:O = CSE Lab, CN = CSE Lab Intermediate CA
   i:O = CSE Lab, CN = CSE Lab Root CA
   a:PKEY: rsaEncryption, 4096 (bit); sigalg: RSA-SHA256
   v:NotBefore: Oct 29 10:24:37 2026 GMT; NotAfter: Oct 28 10:24:37 2031 GMT
-----BEGIN CERTIFICATE-----
MIIFQjCCAyqgAwIBAgICEAAwDQYJKoZIhvcNAQELBQAwLDEQMA4GA1UECgwHQ1NF
IExhYjEYMBYGA1UEAwwPQ1NFIExhYiBSb290IENBMB4XDTI2MTAyOTEwMjQzN1oX
DTMxMTAyODEwMjQzN1owNDEQMA4GA1UECgwHQ1NFIExhYjEgMB4GA1UEAwwXQ1NF
IExhYiBJbnRlcm1lZGlhdGUgQ0EwggIiMA0GCSqGSIb3DQEBAQUAA4ICDwAwggIK
AoICAQCDG+HmzE4IITDbaT2zCDwlLGv6QVAQpF8vSghu0C/1vzm8ovfw65nTw3+I
fO4F3UT7CdjTc7lKjBtzZP7ADu+ZMJtJ9dRxNly6AUijLlXYduS5Q2YXJq5rLCnc
B+LZd03zcvsMqT5tzoblKdT2Uj9dM3XP8+i3+O7lTLY9VxgJCLJ5HLvk3grodn0T
Bh6DcrmJbFnESWFYw4zxDVsFARB/0O+kf3M8d2w+pMc/SOry1AKh1nkEc2xhCLSy
BKP3HreN7K/fJryne+jtwjbirrONDySGSvfHQ380cCAox7mPj9g9FGqvcL1hilwB
rNiqU9eXGvB03hFc4tT6klfsoEpvFdKWKh9c/2zabTKE1wNJi2MAydErjX6QC3PZ
RDF6yqXZQ7Abftjk6ej9o76D6fyMMmQ+VhERrBVJpM7sPpd3Qsj3110TSkifIbPf
0Guedgnn/y+g0rvHmqLPGztQ8hvnJcYA3yIwEKAyrnEm8ueswJZYplF6xo2HuNwL
```

Figure 21: Part D.2: openssl s_client -showcerts (chain and acceptable client CA names) (2/2)



Answers: Task D — Mutual TLS with nginx

1. Outputs and status.

- Alice: `Hello CN=alice,O=CSE Lab | verify=SUCCESS | proto=TLSv1.3`
- No certificate: **HTTP 400 Bad Request**, "No required SSL certificate was sent". The TLS handshake itself completes; then nginx, with `ssl_verify_client on`, sees that no client certificate was sent and refuses the request with 400.

2. What identifies the client? The variable **`$ssl_client_s_dn`** (printed as CN=alice,O=CSE Lab). In error.log a missing certificate appears as *"client sent no required SSL certificate"* and a bad one as *"client SSL certificate verify error: (N:reason)"*. (The default access.log does not show the DN.)

3. Why ssl_client_certificate and ssl_verify_depth 2? `ssl_client_certificate` gives nginx the **trusted CA chain** to check client certificates against; without it every client certificate would fail. `ssl_verify_depth` is the longest chain nginx will follow: client (0) → Intermediate (1) → Root (2), so 2 fits our chain.

Part E — Revocation (CRL and OCSP)

Bob's certificate (serial 1002) is revoked with reason keyCompromise, both CAs publish a CRL, and nginx enforces the combined CRL. An OCSP responder answers live status queries from the Intermediate's index.txt.

Figure 22: Part E.1: revoke Bob, generate both CRLs and the combined CRL (1/2)

```
anush@anusha:~/pki-lab$ openssl ca -config intermediate/openssl.cnf -revoke client/bob.cert.pem -crl_reason keyCompr
omise
Using configuration from intermediate/openssl.cnf
Revoking Certificate 1002.
Database updated
anush@anusha:~/pki-lab$ openssl ca -config intermediate/openssl.cnf -gencrl -out intermediate/crl/intermediate.crl.p
em
Using configuration from intermediate/openssl.cnf
anush@anusha:~/pki-lab$ openssl ca -config root/openssl.cnf -gencrl -out root/crl/root.crl.pem
Using configuration from root/openssl.cnf
anush@anusha:~/pki-lab$ cat intermediate/crl/intermediate.crl.pem root/crl/root.crl.pem > intermediate/crl/combined.
crl.pem
anush@anusha:~/pki-lab$ openssl crl -in intermediate/crl/intermediate.crl.pem -noout -text
Certificate Revocation List (CRL):
        Version 2 (0x1)
        Signature Algorithm: sha256WithRSAEncryption
        Issuer: O = CSE Lab, CN = CSE Lab Intermediate CA
        Last Update: Oct 29 10:24:42 2026 GMT
        Next Update: Nov 28 10:24:42 2026 GMT
        CRL extensions:
            X509v3 CRL Number:
                4096
Revoked Certificates:
    Serial Number: 1002
        Revocation Date: Oct 29 10:24:42 2026 GMT
        CRL entry extensions:
            X509v3 CRL Reason Code:
                Key Compromise
    Signature Algorithm: sha256WithRSAEncryption
    Signature Value:
        00:8f:3c:ad:45:75:01:e1:7e:b7:4e:00:fb:40:cc:16:84:aa:
        55:66:a7:66:60:be:6a:2d:11:9b:04:6e:bb:60:8c:19:36:78:
        4c:26:2e:b8:43:87:5d:0b:4c:96:3a:e7:99:d7:68:7e:e0:4a:
        80:84:11:8b:44:04:7e:0f:9b:fc:82:99:e9:e9:93:c9:ba:85:
        44:d1:b8:df:37:94:3c:6b:d7:1e:b0:4e:be:dd:2e:1a:38:9b:
        65:93:d6:c3:ea:30:9e:27:8e:ca:18:a8:57:a1:c8:d7:ca:ce:
        b2:9e:70:fc:3a:aa:00:b4:44:56:62:77:f8:f8:db:37:eb:b9:
        b2:40:6d:88:6d:3d:14:b4:d7:69:9f:50:0b:7c:f9:b4:89:38:
        75:8e:09:c0:fb:09:b1:04:db:60:2c:30:1a:6b:7f:dc:af:50:
        7e:45:a8:da:81:37:fc:c1:6e:8a:44:aa:43:01:af:7d:d5:9f:
        13:5d:66:a2:05:67:df:d3:3c:15:79:76:ab:de:73:3f:d2:9b:
        13:58:e6:f6:f2:4b:f8:fa:e2:65:98:96:58:4f:4d:eb:a1:88:
        e1:62:25:89:e9:b6:42:cc:2d:06:2a:e8:fc:33:79:03:f8:6c:
        e6:25:6a:59:6f:25:88:47:d1:48:90:f0:b2:f9:52:6c:48:d1:
        f8:1e:a2:67:5f:71:c1:c8:3b:0e:76:7c:7f:63:5f:84:71:e8:
        41:bb:4a:7c:f2:24:e3:dc:6a:1f:ed:10:4e:67:6a:53:4e:02:
        6a:70:52:67:37:7d:71:46:39:80:ef:54:11:0d:de:09:fd:b0:
        69:17:e2:92:e2:2d:c0:df:b5:16:81:8c:3f:d5:40:c4:b2:8a:
        be:d6:f7:6f:61:31:0f:f8:a8:90:d8:2f:80:4b:57:82:ed:4f:
        31:b9:5b:96:14:08:e4:2c:f8:29:cc:d3:c0:c7:df:88:4b:a9:
        b7:46:be:66:8c:3c:25:1e:26:06:2d:b7:31:b1:8b:61:05:58:
        27:fb:10:1d:2a:15:4a:e4:f7:e0:bb:58:d3:08:2f:73:fa:88:
        cc:f1:50:de:5c:b1:42:57:57:3f:14:57:42:9b:7e:42:f5:ad:
        18:9d:94:35:0f:32:69:c0:2b:99:b8:13:fc:03:3a:e3:28:63:
        33:cb:e4:eb:5e:f5:80:23:5e:25:be:49:d1:6e:09:93:88:f5:
        f7:ac:35:e2:c3:86:61:fb:b5:71:8b:05:42:a8:e9:53:b6:d4:
        99:60:67:6a:25:ef:01:bb:a8:f4:18:d1:2d:d7:7b:48:8b:ad:
        06:2b:93:22:13:23:c2:76:44:5f:2e:f0:3c:46:e9:b5:81:0c:
        0a:33:12:10:e9:ce:f5:1a
anush@anusha:~/pki-lab$ openssl crl -in root/crl/root.crl.pem -noout -text
Certificate Revocation List (CRL):
        Version 2 (0x1)
```

Figure 23: Part E.1: revoke Bob, generate both CRLs and the combined CRL (2/2)

```
        Signature Algorithm: sha256WithRSAEncryption
        Issuer: O = CSE Lab, CN = CSE Lab Root CA
        Last Update: Oct 29 10:24:42 2026 GMT
        Next Update: Nov 28 10:24:42 2026 GMT
        CRL extensions:
            X509v3 CRL Number:
                4096
No Revoked Certificates.
    Signature Algorithm: sha256WithRSAEncryption
    Signature Value:
        71:1f:61:fd:09:b4:5c:f7:da:29:29:68:30:d7:73:ff:ca:36:
        16:7c:d5:d9:bd:b9:b5:2c:c6:15:dd:0f:ec:36:42:b2:e9:fb:
        cb:e9:2a:06:dd:45:e9:d0:ae:63:89:cd:78:92:ee:db:03:6d:
        7e:09:46:6e:c1:0f:fa:78:9c:b4:a2:e6:d5:2b:cf:af:32:5e:
        6b:48:5f:c7:45:0b:2e:aa:dc:c7:68:d1:1b:1a:d5:9a:d1:ae:
        a1:6a:39:cd:57:6c:0f:8f:e5:ab:93:96:ad:4c:fb:87:4c:73:
        16:69:45:67:32:f0:ea:95:eb:67:5d:31:e8:31:dd:41:cc:be:
        1c:b1:82:aa:18:7c:6d:ea:6e:66:f5:b5:2a:3b:6b:00:92:2f:
        88:69:58:85:b1:9b:47:e6:ab:40:37:ea:f4:73:62:6a:9c:ce:
        12:fa:e8:f5:45:6c:52:b5:56:cb:02:5a:b9:76:b8:cf:8b:d5:
        e5:d1:41:7d:03:da:6c:69:99:87:28:e7:e0:31:1e:cc:6a:fe:
        10:e1:03:81:d6:a0:df:98:61:12:0f:d7:34:dd:a5:55:95:fc:
        16:7c:e9:84:26:d8:30:88:21:23:c3:4e:d5:22:e7:f1:0f:bf:
        67:d3:9c:2f:73:95:52:3a:a5:6d:49:67:e8:3e:59:dd:89:a4:
        c6:a7:c7:84:eb:59:11:af:f7:0a:ec:d9:34:e3:7c:e9:14:bb:
        f6:34:08:51:92:20:c6:da:ef:fc:bd:ae:93:18:03:70:91:49:
        af:67:a8:dc:63:65:b0:57:60:87:85:34:87:6d:ba:47:23:65:
        ed:82:f8:a7:31:3a:8a:6c:fe:7e:07:01:61:72:76:f3:c1:42:
        a2:36:8d:74:e5:a4:7d:74:2a:53:06:fe:be:4c:04:51:b2:f2:
        56:58:b6:f8:f6:77:88:85:fc:f3:83:2f:5a:2a:9c:21:63:f2:
        70:70:8e:95:ea:b4:9b:d7:c7:79:26:f6:e0:12:dc:88:c4:59:
        cb:6c:4c:f4:13:79:c5:ca:39:db:52:6e:1c:f0:3c:fb:01:12:
        89:95:19:0c:41:52:7c:b1:7d:6a:b5:51:ab:48:09:13:08:c8:
        a0:27:74:9d:17:bc:1c:07:98:5c:df:30:0c:46:31:bb:4e:4a:
        e6:bc:bc:fd:5f:e1:61:52:67:c6:a9:4a:a0:94:d5:0a:69:d0:
        3c:8d:80:49:c7:15:ab:cb:b4:2f:34:90:94:a9:e0:ce:21:eb:
        91:93:f7:a4:20:ab:9a:fa:fa:82:46:91:39:db:fd:42:c8:98:
        4e:a2:33:a3:df:2e:83:cb:c1:8b:98:30:bc:05:18:5d:41:de:
        ae:34:c1:de:b3:fa:05:c7
anush@anusha:~/pki-lab$ cat intermediate/index.txt
V       271029102438Z           1000    unknown /O=CSE Lab/CN=server.lab.local
V       271029102438Z           1001    unknown /O=CSE Lab/CN=alice
R       271029102439Z   261029102442Z,keyCompromise     1002    unknown /O=CSE Lab/CN=bob
anush@anusha:~/pki-lab$
```

Figure 24: Part E.1: CRL check with OpenSSL (Bob revoked, Alice OK)

```
anush@anusha:~/pki-lab$ openssl verify -crl_check -CAfile intermediate/certs/ca-chain.cert.pem \
>   -CRLfile intermediate/crl/combined.crl.pem client/bob.cert.pem
O = CSE Lab, CN = bob
error 23 at 0 depth lookup: certificate revoked
error client/bob.cert.pem: verification failed
anush@anusha:~/pki-lab$ openssl verify -crl_check -CAfile intermediate/certs/ca-chain.cert.pem \
>   -CRLfile intermediate/crl/combined.crl.pem client/alice.cert.pem
client/alice.cert.pem: OK
anush@anusha:~/pki-lab$
```

Figure 25: Part E.2: ssl_crl in nginx: Alice accepted, Bob rejected

```
anush@anusha:~/pki-lab$ sed -i 's#^    ssl_verify_depth        2;#    ssl_verify_depth        2;\n    ssl_crl
         /home/anush/pki-lab/intermediate/crl/combined.crl.pem;#' nginx/nginx.conf
anush@anusha:~/pki-lab$ grep -n ssl_ nginx/nginx.conf
10:    ssl_certificate         /home/anush/pki-lab/server/server.fullchain.pem;
11:    ssl_certificate_key     /home/anush/pki-lab/server/server.key.pem;
12:    ssl_protocols           TLSv1.2 TLSv1.3;
14:    ssl_client_certificate  /home/anush/pki-lab/intermediate/certs/ca-chain.cert.pem;
15:    ssl_verify_client       on;
16:    ssl_verify_depth        2;
17:    ssl_crl                 /home/anush/pki-lab/intermediate/crl/combined.crl.pem;
20:      return 200 'Hello $ssl_client_s_dn | verify=$ssl_client_verify | proto=$ssl_protocol';
anush@anusha:~/pki-lab$ nginx -p $PKI/nginx -c $PKI/nginx/nginx.conf -s reload
2026/10/29 10:24:42 [notice] 1284#1284: signal process started
anush@anusha:~/pki-lab$ curl --cacert intermediate/certs/ca-chain.cert.pem \
>   --cert client/alice.cert.pem --key client/alice.key.pem https://server.lab.local:8443/
Hello CN=alice,O=CSE Lab | verify=SUCCESS | proto=TLSv1.3
anush@anusha:~/pki-lab$ echo
anush@anusha:~/pki-lab$ curl --cacert intermediate/certs/ca-chain.cert.pem \
>   --cert client/bob.cert.pem --key client/bob.key.pem https://server.lab.local:8443/
<html>
<head><title>400 The SSL certificate error</title></head>
<body>
<center><h1>400 Bad Request</h1></center>
<center>The SSL certificate error</center>
<hr><center>nginx/1.24.0 (Ubuntu)</center>
</body>
</html>
anush@anusha:~/pki-lab$ tail -n +22 nginx/error.log
2026/10/29 10:24:44 [info] 1285#1285: *5 client 127.0.0.1 closed keepalive connection
2026/10/29 10:24:44 [info] 1285#1285: *6 client SSL certificate verify error: (23:certificate revoked) while reading
 client request headers, client: 127.0.0.1, server: server.lab.local, request: "GET / HTTP/1.1", host: "server.lab.l
ocal:8443"
anush@anusha:~/pki-lab$
```

Figure 26: Part E.2: without the Root CRL even Alice fails (error 3), then restored

```
anush@anusha:~/pki-lab$ cp intermediate/crl/combined.crl.pem intermediate/crl/combined.crl.pem.bak
anush@anusha:~/pki-lab$ cp intermediate/crl/intermediate.crl.pem intermediate/crl/combined.crl.pem
anush@anusha:~/pki-lab$ nginx -p $PKI/nginx -c $PKI/nginx/nginx.conf -s reload
2026/10/29 10:24:44 [notice] 1346#1346: signal process started
anush@anusha:~/pki-lab$ curl --cacert intermediate/certs/ca-chain.cert.pem \
>   --cert client/alice.cert.pem --key client/alice.key.pem https://server.lab.local:8443/
<html>
<head><title>400 The SSL certificate error</title></head>
<body>
<center><h1>400 Bad Request</h1></center>
<center>The SSL certificate error</center>
<hr><center>nginx/1.24.0 (Ubuntu)</center>
</body>
</html>
anush@anusha:~/pki-lab$ tail -n +35 nginx/error.log
2026/10/29 10:24:46 [info] 1347#1347: *7 client SSL certificate verify error: (3:unable to get certificate CRL) whil
e reading client request headers, client: 127.0.0.1, server: server.lab.local, request: "GET / HTTP/1.1", host: "ser
ver.lab.local:8443"
anush@anusha:~/pki-lab$ mv intermediate/crl/combined.crl.pem.bak intermediate/crl/combined.crl.pem
anush@anusha:~/pki-lab$ nginx -p $PKI/nginx -c $PKI/nginx/nginx.conf -s reload
2026/10/29 10:24:46 [notice] 1383#1383: signal process started
anush@anusha:~/pki-lab$ curl --cacert intermediate/certs/ca-chain.cert.pem \
>   --cert client/alice.cert.pem --key client/alice.key.pem https://server.lab.local:8443/
Hello CN=alice,O=CSE Lab | verify=SUCCESS | proto=TLSv1.3
anush@anusha:~/pki-lab$ echo
anush@anusha:~/pki-lab$
```

Figure 27: Part E.3 (terminal 2): OCSP responder on port 2560

```
anush@anusha:~/pki-lab$ export PKI=$HOME/pki-lab; cd $PKI
anush@anusha:~/pki-lab$
```

Figure 28: Part E.3: OCSP query for Alice - response data

```
anush@anusha:~/pki-lab$ openssl ocsp -issuer intermediate/certs/intermediate.cert.pem \
>   -cert client/alice.cert.pem -url http://127.0.0.1:2560 \
>   -CAfile intermediate/certs/ca-chain.cert.pem -resp_text
OCSP Response Data:
    OCSP Response Status: successful (0x0)
    Response Type: Basic OCSP Response
    Version: 1 (0x0)
    Responder Id: O = CSE Lab, CN = CSE Lab Intermediate CA
    Produced At: Oct 29 10:24:49 2026 GMT
    Responses:
    Certificate ID:
      Hash Algorithm: sha1
      Issuer Name Hash: 608BB053FB4F26D893AF694E08BCF348E86FC675
      Issuer Key Hash: BD83593B51569C24A6A1A86E5B4B04691B9A71EA
      Serial Number: 1001
    Cert Status: good
    This Update: Oct 29 10:24:49 2026 GMT

    Response Extensions:
anush@anusha:~/pki-lab$
```

Figure 29: Part E.3: OCSP query for Alice - end of output (status good)

```
bBCtIIaHoQoBKPE9INKOs7x0X48vepill3DbXkUyZmzkDWTMHVn+Diknxvknfpzk
BTbruLxE/Pn+Tz0zqAolw1fEbVppnqu7YiRU3+4mQnC9L1/OjEdTbyIgatDnHhB7
lmSqLtCL
-----END CERTIFICATE-----
Response verify OK
client/alice.cert.pem: good
        This Update: Oct 29 10:24:49 2026 GMT
anush@anusha:~/pki-lab$
```

Figure 30: Part E.3: OCSP query for Bob - response data (revoked, keyCompromise)

```
anush@anusha:~/pki-lab$ openssl ocsp -issuer intermediate/certs/intermediate.cert.pem \
>   -cert client/bob.cert.pem -url http://127.0.0.1:2560 \
>   -CAfile intermediate/certs/ca-chain.cert.pem -resp_text
OCSP Response Data:
    OCSP Response Status: successful (0x0)
    Response Type: Basic OCSP Response
    Version: 1 (0x0)
    Responder Id: O = CSE Lab, CN = CSE Lab Intermediate CA
    Produced At: Oct 29 10:24:49 2026 GMT
    Responses:
    Certificate ID:
      Hash Algorithm: sha1
      Issuer Name Hash: 608BB053FB4F26D893AF694E08BCF348E86FC675
      Issuer Key Hash: BD83593B51569C24A6A1A86E5B4B04691B9A71EA
      Serial Number: 1002
    Cert Status: revoked
    Revocation Time: Oct 29 10:24:42 2026 GMT
    Revocation Reason: keyCompromise (0x1)
    This Update: Oct 29 10:24:49 2026 GMT

    Response Extensions:
        OCSP Nonce:
anush@anusha:~/pki-lab$
```

Figure 31: Part E.3: OCSP query for Bob - end of output (status revoked)

```
bBCtIIaHoQoBKPE9INKOs7x0X48vepill3DbXkUyZmzkDWTMHVn+Diknxvknfpzk
BTbruLxE/Pn+Tz0zqAolw1fEbVppnqu7YiRU3+4mQnC9L1/OjEdTbyIgatDnHhB7
lmSqLtCL
-----END CERTIFICATE-----
Response verify OK
client/bob.cert.pem: revoked
        This Update: Oct 29 10:24:49 2026 GMT
        Reason: keyCompromise
        Revocation Time: Oct 29 10:24:42 2026 GMT
anush@anusha:~/pki-lab$
```

Figure 32: Task E: OCSP URL in the Authority Information Access extension

```
anush@anusha:~/pki-lab$ openssl x509 -in server/server.cert.pem -noout -ext authorityInfoAccess
Authority Information Access:
    OCSP - URI:http://127.0.0.1:2560
anush@anusha:~/pki-lab$
```

Answers: Task E — Revocation

1. Status.

- Alice: **good**
- Bob: **revoked**, revocation time **Oct 29 10:24:42 2026 GMT**, reason **keyCompromise**

2. CRL vs OCSP

	CRL	OCSP
Freshness	Only as new as the last download (valid until Next Update, 30 days here)	Near real time (answers from the live database)
Size	Grows with every revoked certificate	One small request and response per certificate
Privacy	Good: the CA doesn't learn which certificate you check	Weak: the responder sees which certificate (site) you check
Latency	One download, then local look-ups	Extra round trip during connection (unless stapled)
Failure	nginx **hard-fails** if the CRL is missing (error 3)	Browsers usually **soft-fail** if the responder is down

3. OCSP stapling. The **server** fetches a signed, time-stamped OCSP response for its own certificate and attaches it to the TLS handshake. Clients don't contact the CA, which fixes OCSP's **privacy leak**, **extra latency** and **dependence on the responder** being up. It can't be forged, because the CA signs it.

4. Where is the OCSP URL? In the **Authority Information Access** extension: `OCSP - URI:http://127.0.0.1:2560`.

Part F — TLS handshake analysis

Both handshakes were captured on the loopback interface (lo) with the capture filter tcp port 8443, using Alice's certificate. For TLS 1.3 the session secrets were written to keys.log (SSLKEYLOGFILE) during the same captured connection, so that the capture can be decrypted.

F.1 TLS 1.2

Figure 33: Part F.1 (terminal 2): tshark capture on lo, tcp port 8443 (TLS 1.2)

```
anush@anusha:~/pki-lab$ sudo tshark -i lo -f "tcp port 8443" -w captures/tls12.pcapng
Capturing on 'Loopback: lo'
15 packets captured
anush@anusha:~/pki-lab$
```

Figure 34: Part F.1: TLS 1.2 handshake - ClientHello, ServerHello, Certificate, CertificateRequest, CertificateVerify (1/3)

```
anush@anusha:~/pki-lab$ curl --tlsv1.2 --tls-max 1.2 --cacert intermediate/certs/ca-chain.cert.pem \
>   --cert client/alice.cert.pem --key client/alice.key.pem https://server.lab.local:8443/
Hello CN=alice,O=CSE Lab | verify=SUCCESS | proto=TLSv1.2
anush@anusha:~/pki-lab$ tshark -r captures/tls12.pcapng -Y tls.handshake
    4 0.008441405    127.0.0.1 → 127.0.0.1    TLSv1 293 Client Hello (SNI=server.lab.local)
    6 0.012909438    127.0.0.1 → 127.0.0.1    TLSv1.2 3288 Server Hello, Certificate, Server Key Exchange, Certificate Request, Server Hello Done
    8 0.021829102    127.0.0.1 → 127.0.0.1    TLSv1.2 2999 Certificate, Client Key Exchange, Certificate Verify, Change Cipher Spec, Encrypted Handshake Message
    9 0.024233269    127.0.0.1 → 127.0.0.1    TLSv1.2 117 Change Cipher Spec, Encrypted Handshake Message
anush@anusha:~/pki-lab$ tshark -r captures/tls12.pcapng -Y 'tls.handshake.type == 1' -V | grep -E 'Server Name:|Cipher Suites \(|Cipher Suites Length'
            Cipher Suites Length: 56
            Cipher Suites (28 suites)
                    Server Name: server.lab.local
anush@anusha:~/pki-lab$ tshark -r captures/tls12.pcapng -Y 'tls.handshake.type == 1' -T fields -e tls.handshake.ciphersuite | tr ',' '\n' | head -5
0xc02c
0xc030
0x009f
0xcca9
0xcca8
anush@anusha:~/pki-lab$ tshark -r captures/tls12.pcapng -Y 'tls.handshake.type == 2' -V | grep -E 'Handshake Type: Server Hello|Cipher Suite:'
            Handshake Type: Server Hello (2)
            Cipher Suite: TLS_ECDHE_RSA_WITH_AES_256_GCM_SHA384 (0xc030)
            Handshake Type: Server Hello Done (14)
anush@anusha:~/pki-lab$ tshark -r captures/tls12.pcapng -Y 'tls.handshake.type == 11'
    6 0.012909438    127.0.0.1 → 127.0.0.1    TLSv1.2 3288 Server Hello, Certificate, Server Key Exchange, Certificate Request, Server Hello Done
    8 0.021829102    127.0.0.1 → 127.0.0.1    TLSv1.2 2999 Certificate, Client Key Exchange, Certificate Verify, Change Cipher Spec, Encrypted Handshake Message
anush@anusha:~/pki-lab$ tshark -r captures/tls12.pcapng -Y 'tls.handshake.type == 11' -V | grep -E 'Handshake Type|Certificate Length|rdnSequence|utcTime|dNSName|iPAddress|KeyPurposeId|accessLocation|uniformResourceIdentifier'
            Handshake Type: Server Hello (2)
            Handshake Type: Certificate (11)
                Certificate Length: 1262
                        issuer: rdnSequence (0)
                            rdnSequence: 2 items (id-at-commonName=CSE Lab Intermediate CA,id-at-organizationName=CSE Lab)
                            notBefore: utcTime (0)
                                utcTime: 2026-10-29 10:24:38 (UTC)
                            notAfter: utcTime (0)
                                utcTime: 2027-10-29 10:24:38 (UTC)
                        subject: rdnSequence (0)
                            rdnSequence: 2 items (id-at-commonName=server.lab.local,id-at-organizationName=CSE Lab)
                                        directoryName: rdnSequence (0)
                                            rdnSequence: 2 items (id-at-commonName=CSE Lab Root CA,id-at-organizationName=CSE Lab)
                                    KeyPurposeId: 1.3.6.1.5.5.7.3.1 (id-kp-serverAuth)
                                    GeneralName: dNSName (2)
                                        dNSName: server.lab.local
                                    GeneralName: dNSName (2)
                                        dNSName: localhost
                                    GeneralName: iPAddress (7)
                                        iPAddress: 127.0.0.1
                                        accessLocation: 6
                                            uniformResourceIdentifier: http://127.0.0.1:2560
                Certificate Length: 1350
                        issuer: rdnSequence (0)
                            rdnSequence: 2 items (id-at-commonName=CSE Lab Root CA,id-at-organizationName=CSE Lab)
                            notBefore: utcTime (0)
```

Figure 35: Part F.1: TLS 1.2 handshake - ClientHello, ServerHello, Certificate, CertificateRequest, CertificateVerify (2/3)

```
                            utcTime: 2026-10-29 10:24:37 (UTC)
                        notAfter: utcTime (0)
                            utcTime: 2031-10-28 10:24:37 (UTC)
                    subject: rdnSequence (0)
                        rdnSequence: 2 items (id-at-commonName=CSE Lab Intermediate CA,id-at-organizationName=CS
E Lab)
            Handshake Type: Server Key Exchange (12)
            Handshake Type: Certificate Request (13)
            Handshake Type: Server Hello Done (14)
            Handshake Type: Certificate (11)
                Certificate Length: 1203
                        issuer: rdnSequence (0)
                            rdnSequence: 2 items (id-at-commonName=CSE Lab Intermediate CA,id-at-organizationName=CS
E Lab)
                            notBefore: utcTime (0)
                                utcTime: 2026-10-29 10:24:38 (UTC)
                            notAfter: utcTime (0)
                                utcTime: 2027-10-29 10:24:38 (UTC)
                        subject: rdnSequence (0)
                            rdnSequence: 2 items (id-at-commonName=alice,id-at-organizationName=CSE Lab)
                                        directoryName: rdnSequence (0)
                                            rdnSequence: 2 items (id-at-commonName=CSE Lab Root CA,id-at-organiz
ationName=CSE Lab)
                                    KeyPurposeId: 1.3.6.1.5.5.7.3.2 (id-kp-clientAuth)
                                        accessLocation: 6
                                            uniformResourceIdentifier: http://127.0.0.1:2560
                Certificate Length: 1350
                        issuer: rdnSequence (0)
                            rdnSequence: 2 items (id-at-commonName=CSE Lab Root CA,id-at-organizationName=CSE Lab)
                            notBefore: utcTime (0)
                                utcTime: 2026-10-29 10:24:37 (UTC)
                            notAfter: utcTime (0)
                                utcTime: 2031-10-28 10:24:37 (UTC)
                        subject: rdnSequence (0)
                            rdnSequence: 2 items (id-at-commonName=CSE Lab Intermediate CA,id-at-organizationName=CS
E Lab)
            Handshake Type: Client Key Exchange (16)
            Handshake Type: Certificate Verify (15)
anush@anusha:~/pki-lab$ tshark -r captures/tls12.pcapng -Y 'tls.handshake.type == 13 || tls.handshake.type == 15'
    6 0.012909438    127.0.0.1 → 127.0.0.1    TLSv1.2 3288 Server Hello, Certificate, Server Key Exchange, Certifica
te Request, Server Hello Done
    8 0.021829102    127.0.0.1 → 127.0.0.1    TLSv1.2 2999 Certificate, Client Key Exchange, Certificate Verify, Cha
nge Cipher Spec, Encrypted Handshake Message
anush@anusha:~/pki-lab$ tshark -r captures/tls12.pcapng -Y 'tls.handshake.type == 13' -V | grep -E 'Handshake Type|D
istinguished Name|rdnSequence'
            Handshake Type: Server Hello (2)
            Handshake Type: Certificate (11)
                        issuer: rdnSequence (0)
                            rdnSequence: 2 items (id-at-commonName=CSE Lab Intermediate CA,id-at-organizationName=CS
E Lab)
                        subject: rdnSequence (0)
                            rdnSequence: 2 items (id-at-commonName=server.lab.local,id-at-organizationName=CSE Lab)
                                        directoryName: rdnSequence (0)
                                            rdnSequence: 2 items (id-at-commonName=CSE Lab Root CA,id-at-organiz
ationName=CSE Lab)
                        issuer: rdnSequence (0)
                            rdnSequence: 2 items (id-at-commonName=CSE Lab Root CA,id-at-organizationName=CSE Lab)
                        subject: rdnSequence (0)
                            rdnSequence: 2 items (id-at-commonName=CSE Lab Intermediate CA,id-at-organizationName=CS
E Lab)
            Handshake Type: Server Key Exchange (12)
            Handshake Type: Certificate Request (13)
```

Figure 36: Part F.1: TLS 1.2 handshake - ClientHello, ServerHello, Certificate, CertificateRequest, CertificateVerify (3/3)

```
            Distinguished Names Length: 104
            Distinguished Names (104 bytes)
                Distinguished Name Length: 54
                Distinguished Name: (id-at-commonName=CSE Lab Intermediate CA,id-at-organizationName=CSE Lab)
                Distinguished Name Length: 46
                Distinguished Name: (id-at-commonName=CSE Lab Root CA,id-at-organizationName=CSE Lab)
            Handshake Type: Server Hello Done (14)
anush@anusha:~/pki-lab$ tshark -r captures/tls12.pcapng -Y 'tls.handshake.type == 15' -V | grep -E 'Handshake Type|S
ignature Algorithm|Signature length'
            Handshake Type: Certificate (11)
            Handshake Type: Client Key Exchange (16)
            Handshake Type: Certificate Verify (15)
            Signature Algorithm: rsa_pss_rsae_sha256 (0x0804)
            Signature length: 256
anush@anusha:~/pki-lab$
```

Figure 37: Wireshark: tls12.pcapng with display filter tls.handshake

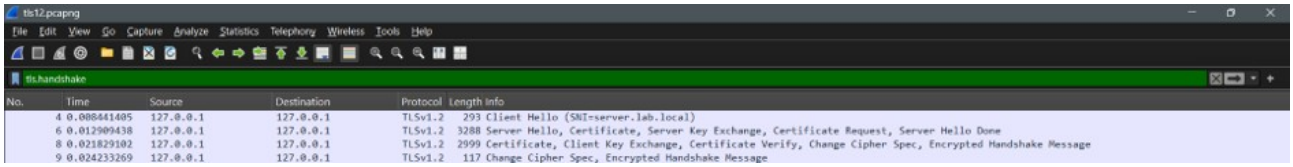


Figure 38: Wireshark: frame 4, Client Hello (TLS 1.2, 28 cipher suites)

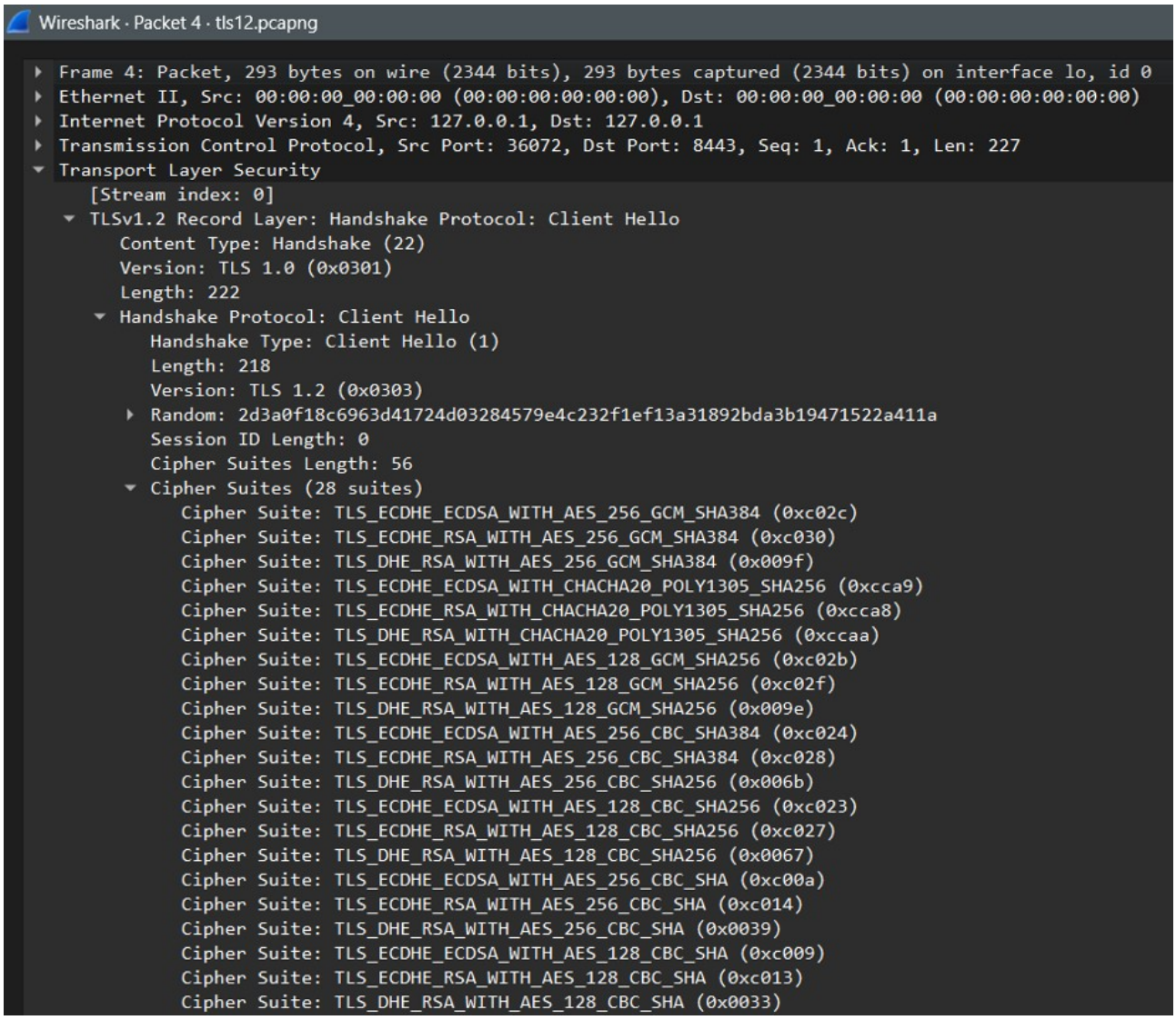


Figure 39: Wireshark view: frame 6, Server Hello - selected cipher suite

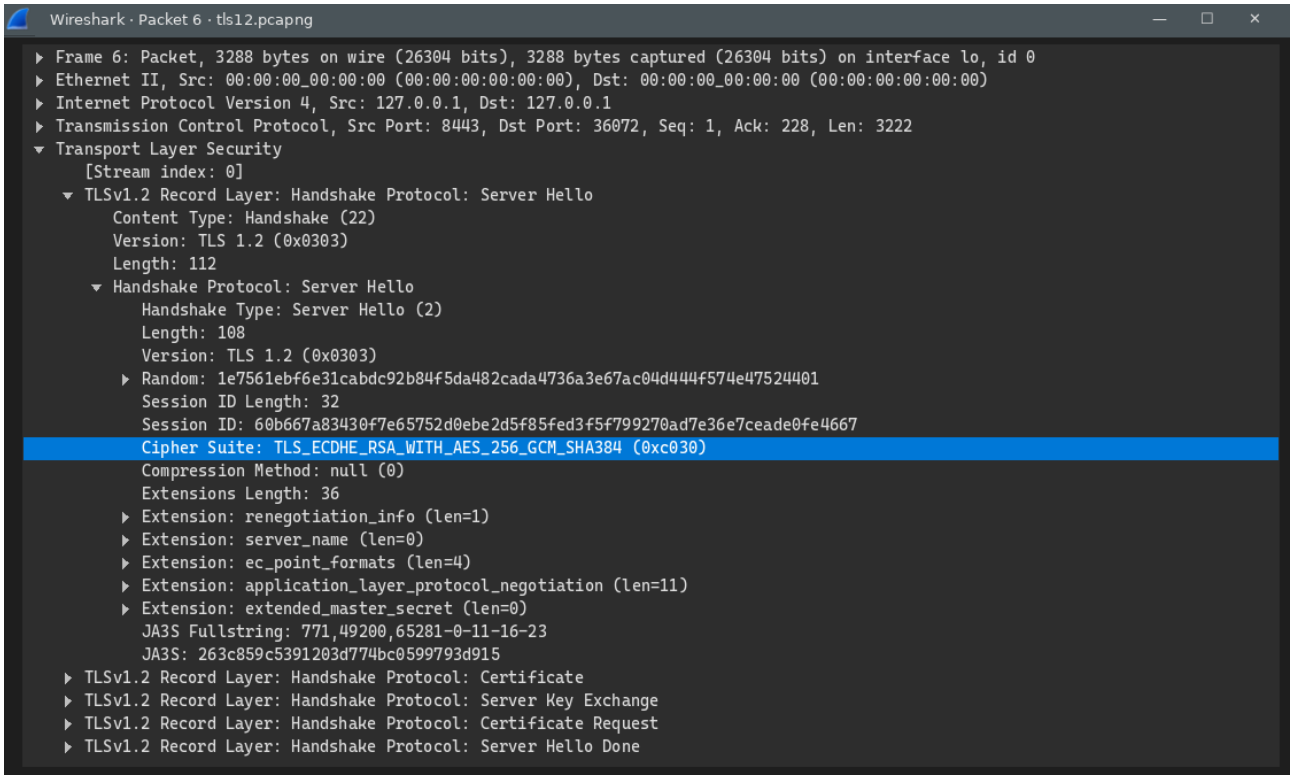


Figure 40: Wireshark view: frame 6, server Certificate - issuer, validity, subject, extensions (EKU, SAN, OCSP URL)



Figure 41: Wireshark view: frame 6, Certificate Request with the acceptable CA names

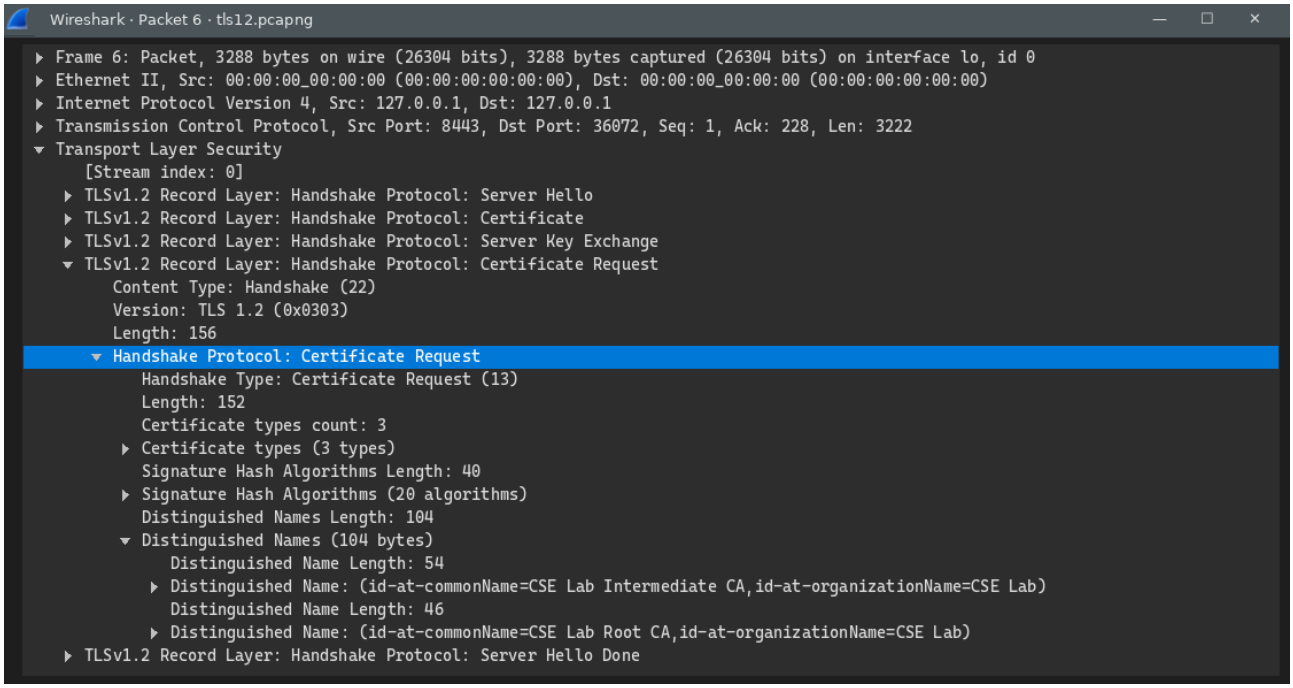
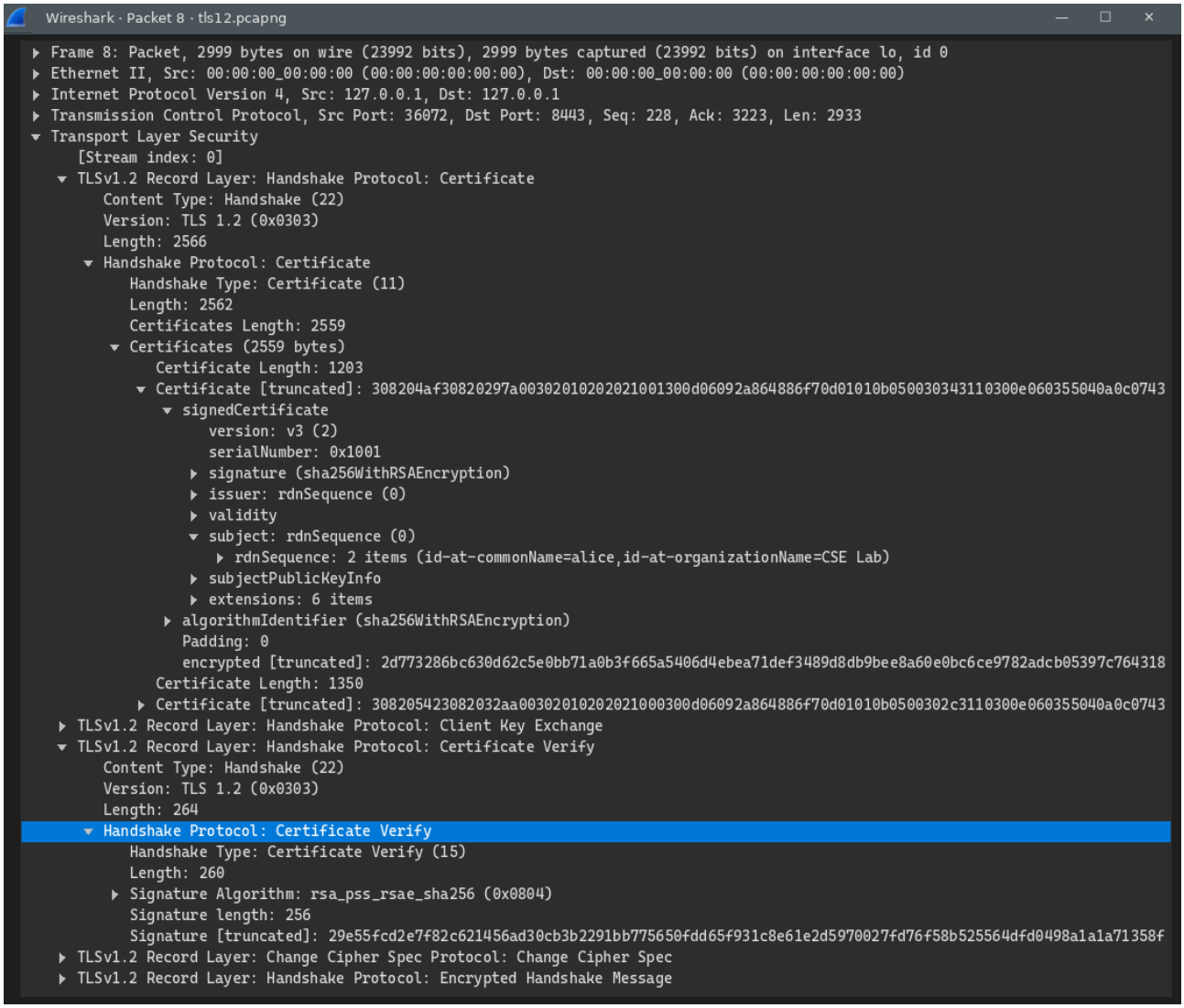


Figure 42: Wireshark view: frame 8, client Certificate (CN=alice) and Certificate Verify



F.2 TLS 1.3 without the key log

Figure 43: Part F.2 (terminal 2): tshark capture on lo, tcp port 8443 (TLS 1.3)

```
anush@anusha:~/pki-lab$ sudo tshark -i lo -f "tcp port 8443" -w captures/tls13.pcapng
Capturing on 'Loopback: lo'
18 packets captured
anush@anusha:~/pki-lab$
```

Figure 44: Part F.2: TLS 1.3 handshake without the key log - everything after ServerHello is encrypted

```
anush@anusha:~/pki-lab$ export SSLKEYLOGFILE=$PKI/captures/keys.log
anush@anusha:~/pki-lab$ curl --tlsv1.3 --cacert intermediate/certs/ca-chain.cert.pem \
>   --cert client/alice.cert.pem --key client/alice.key.pem https://server.lab.local:8443/
Hello CN=alice,O=CSE Lab | verify=SUCCESS | proto=TLSv1.3
anush@anusha:~/pki-lab$ cut -d" " -f1 captures/keys.log
SERVER_HANDSHAKE_TRAFFIC_SECRET
EXPORTER_SECRET
SERVER_TRAFFIC_SECRET_0
CLIENT_HANDSHAKE_TRAFFIC_SECRET
CLIENT_TRAFFIC_SECRET_0
anush@anusha:~/pki-lab$ tshark -r captures/tls13.pcapng -Y tls.handshake
    4 0.005792596    127.0.0.1 → 127.0.0.1    TLSv1 583 Client Hello (SNI=server.lab.local)
    6 0.007646621    127.0.0.1 → 127.0.0.1    TLSv1.3 3435 Server Hello, Change Cipher Spec, Application Data, Appli
cation Data, Application Data, Application Data, Application Data
anush@anusha:~/pki-lab$ tshark -r captures/tls13.pcapng -Y tls
    4 0.005792596    127.0.0.1 → 127.0.0.1    TLSv1 583 Client Hello (SNI=server.lab.local)
    6 0.007646621    127.0.0.1 → 127.0.0.1    TLSv1.3 3435 Server Hello, Change Cipher Spec, Application Data, Appli
cation Data, Application Data, Application Data, Application Data
    8 0.011264088    127.0.0.1 → 127.0.0.1    TLSv1.3 3025 Change Cipher Spec, Application Data, Application Data, A
pplication Data
    9 0.011609148    127.0.0.1 → 127.0.0.1    TLSv1.3 172 Application Data
   11 0.013447144    127.0.0.1 → 127.0.0.1    TLSv1.3 1569 Application Data
   12 0.013932191    127.0.0.1 → 127.0.0.1    TLSv1.3 1569 Application Data
   14 0.014095215    127.0.0.1 → 127.0.0.1    TLSv1.3 302 Application Data
   15 0.014586131    127.0.0.1 → 127.0.0.1    TLSv1.3 90 Application Data
anush@anusha:~/pki-lab$ tshark -r captures/tls13.pcapng -Y 'tls.handshake.type == 11'
anush@anusha:~/pki-lab$ tshark -r captures/tls13.pcapng -Y 'tls.handshake.type == 2' -V | grep -E 'Cipher Suite:|Sup
ported Version:|Group:'
            Cipher Suite: TLS_AES_256_GCM_SHA384 (0x1302)
                Supported Version: TLS 1.3 (0x0304)
                    Key Share Entry: Group: x25519, Key Exchange length: 32
                        Group: x25519 (29)
anush@anusha:~/pki-lab$
```

Figure 45: Wireshark view: tls13.pcapng, filter tls - after the Server Hello everything is Application Data

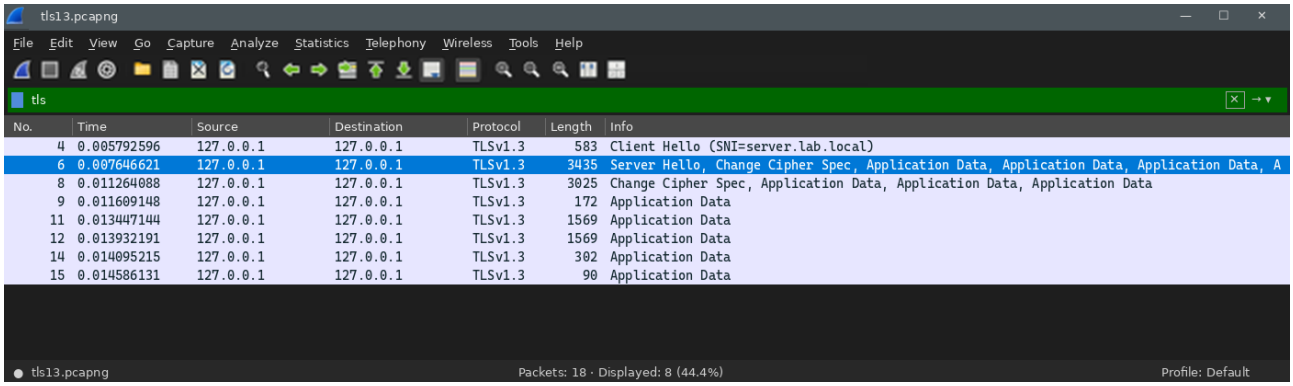
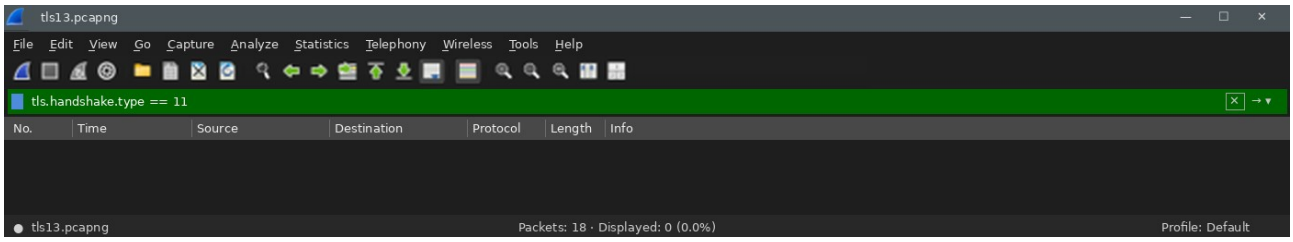


Figure 46: Wireshark view: tls13.pcapng, filter tls.handshake.type == 11 - no Certificate message is visible



F.3 TLS 1.3 decrypted with keys.log

Figure 47: Part F.3: TLS 1.3 handshake decrypted with keys.log (1/2)

```
anush@anusha:~/pki-lab$ tshark -r captures/tls13.pcapng -o tls.keylog_file:captures/keys.log -Y tls.handshake
    4 0.005792596    127.0.0.1 → 127.0.0.1    TLSv1 583 Client Hello (SNI=server.lab.local)
    6 0.007646621    127.0.0.1 → 127.0.0.1    TLSv1.3 3435 Server Hello, Change Cipher Spec, Encrypted Extensions, C
ertificate Request, Certificate, Certificate Verify, Finished
    8 0.011264088    127.0.0.1 → 127.0.0.1    TLSv1.3 3025 Change Cipher Spec, Certificate, Certificate Verify, Fini
shed
   11 0.013447144    127.0.0.1 → 127.0.0.1    TLSv1.3 1569 New Session Ticket
   12 0.013932191    127.0.0.1 → 127.0.0.1    TLSv1.3 1569 New Session Ticket
anush@anusha:~/pki-lab$ tshark -r captures/tls13.pcapng -o tls.keylog_file:captures/keys.log -Y 'tls.handshake.type
== 11'
    6 0.007646621    127.0.0.1 → 127.0.0.1    TLSv1.3 3435 Server Hello, Change Cipher Spec, Encrypted Extensions, C
ertificate Request, Certificate, Certificate Verify, Finished
    8 0.011264088    127.0.0.1 → 127.0.0.1    TLSv1.3 3025 Change Cipher Spec, Certificate, Certificate Verify, Fini
shed
anush@anusha:~/pki-lab$ tshark -r captures/tls13.pcapng -o tls.keylog_file:captures/keys.log -Y 'tls.handshake.type
== 11' -V | grep -E 'Handshake Type|rdnSequence'
            Handshake Type: Server Hello (2)
            Handshake Type: Encrypted Extensions (8)
            Handshake Type: Certificate Request (13)
            Handshake Type: Certificate (11)
                        issuer: rdnSequence (0)
                            rdnSequence: 2 items (id-at-commonName=CSE Lab Intermediate CA,id-at-organizationName=CS
E Lab)
                        subject: rdnSequence (0)
                            rdnSequence: 2 items (id-at-commonName=server.lab.local,id-at-organizationName=CSE Lab)
                                        directoryName: rdnSequence (0)
                                            rdnSequence: 2 items (id-at-commonName=CSE Lab Root CA,id-at-organiz
ationName=CSE Lab)
                        issuer: rdnSequence (0)
                            rdnSequence: 2 items (id-at-commonName=CSE Lab Root CA,id-at-organizationName=CSE Lab)
                        subject: rdnSequence (0)
                            rdnSequence: 2 items (id-at-commonName=CSE Lab Intermediate CA,id-at-organizationName=CS
E Lab)
            Handshake Type: Certificate Verify (15)
            Handshake Type: Finished (20)
            Handshake Type: Certificate (11)
                        issuer: rdnSequence (0)
                            rdnSequence: 2 items (id-at-commonName=CSE Lab Intermediate CA,id-at-organizationName=CS
E Lab)
                        subject: rdnSequence (0)
                            rdnSequence: 2 items (id-at-commonName=alice,id-at-organizationName=CSE Lab)
                                        directoryName: rdnSequence (0)
                                            rdnSequence: 2 items (id-at-commonName=CSE Lab Root CA,id-at-organiz
ationName=CSE Lab)
                        issuer: rdnSequence (0)
                            rdnSequence: 2 items (id-at-commonName=CSE Lab Root CA,id-at-organizationName=CSE Lab)
                        subject: rdnSequence (0)
                            rdnSequence: 2 items (id-at-commonName=CSE Lab Intermediate CA,id-at-organizationName=CS
E Lab)
            Handshake Type: Certificate Verify (15)
            Handshake Type: Finished (20)
anush@anusha:~/pki-lab$ tshark -r captures/tls13.pcapng -o tls.keylog_file:captures/keys.log -Y 'tls.handshake.type
== 13 || tls.handshake.type == 15'
    6 0.007646621    127.0.0.1 → 127.0.0.1    TLSv1.3 3435 Server Hello, Change Cipher Spec, Encrypted Extensions, C
ertificate Request, Certificate, Certificate Verify, Finished
    8 0.011264088    127.0.0.1 → 127.0.0.1    TLSv1.3 3025 Change Cipher Spec, Certificate, Certificate Verify, Fini
shed
anush@anusha:~/pki-lab$ ls -l captures
total 32
-rw-r--r-- 1 anush anush   938 Oct 29 10:24 keys.log
-rw------- 1 anush anush  8752 Oct 29 10:24 tls12.pcapng
-rw------- 1 anush anush 12448 Oct 29 10:24 tls13.pcapng
```

Figure 48: Part F.3: TLS 1.3 handshake decrypted with keys.log (2/2)

```
anush@anusha:~/pki-lab$ |
```

Figure 49: Wireshark view: tls13.pcapng with keys.log, filter tls.handshake - Certificate, Certificate Verify and Finished decrypted

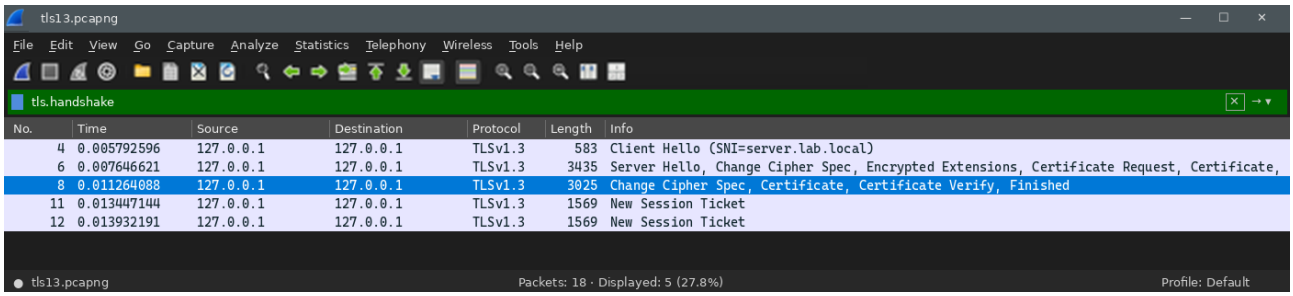
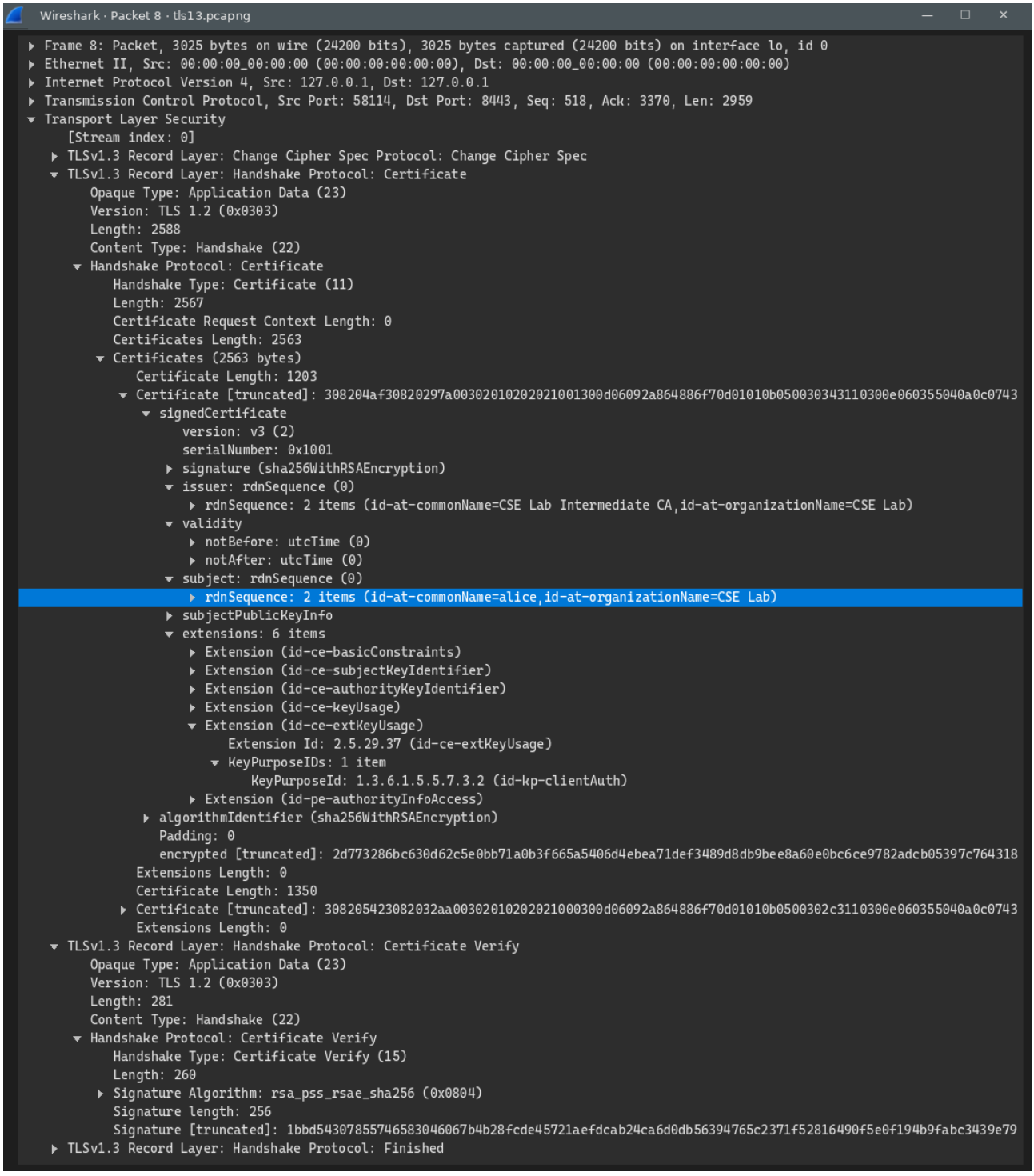


Figure 50: Wireshark view: frame 8 with keys.log - client Certificate (CN=alice, clientAuth) and Certificate Verify



Answers: Task F — TLS handshake analysis

1. TLS 1.2 vs TLS 1.3

	TLS 1.2	**TLS 1.3**
Round trips before data	**2 RTT**	**1 RTT**
Server certificate visible?	**Yes**, plain text	**No**, encrypted (only with keys.log)
Client certificate visible?	**Yes**, plain text	**No**, encrypted

Key-exchange messages	Separate **ServerKeyExchange** and **ClientKeyExchange**	None: key shares sit in the ClientHello/ServerHello **key_share** extension
Cipher suite (my run)	ECDHE-RSA-AES256-GCM-SHA384 (0xc030)	TLS_AES_256_GCM_SHA384 (0x1302), x25519

2. Why is an encrypted client certificate a privacy gain? The client certificate names a **person** (CN=alice, O=CSE Lab). In TLS 1.2 any observer can read it and **identify and track the user** across sessions. In TLS 1.3 an observer still sees IP addresses, the SNI and packet sizes, but **cannot read either certificate**, so they can't tell who logged in.

3. Which message proves the client owns the private key? CertificateVerify: the client signs the handshake transcript with its private key, and the server checks it with the public key in the certificate. (A certificate alone proves nothing, since certificates are public.)

4. Screenshots: see the Part F figures above: ClientHello and ServerHello, the server Certificate, CertificateRequest and CertificateVerify (TLS 1.2); the encrypted TLS 1.3 handshake; and the TLS 1.3 handshake decrypted with keys.log, with the relevant packet or field selected.

Part G — Failure scenarios

Each failure was created on purpose and tested with curl, OpenSSL and the nginx error.log (level info). nginx had ssl_crl enabled from Part E during all scenarios.

Figure 51: Part G.1: expired client certificate (eve)

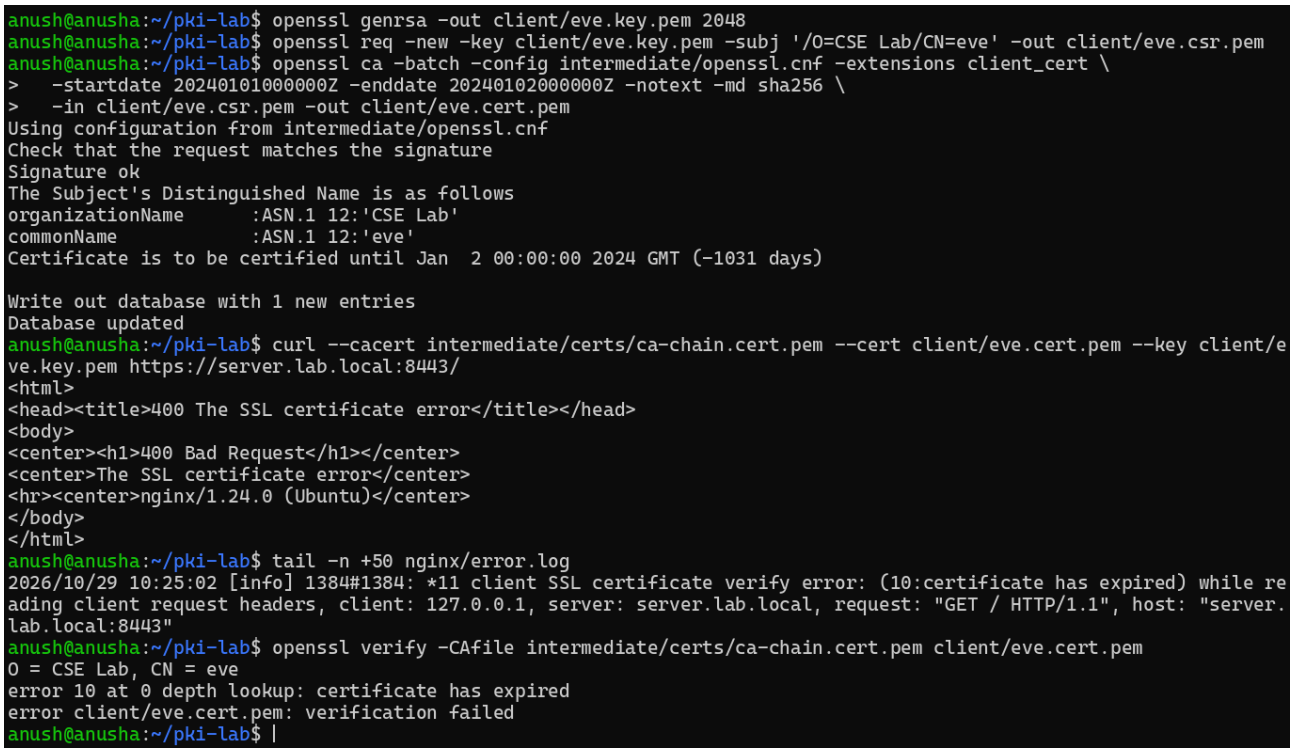


Figure 52: Part G.2: untrusted self-signed client (mallory)

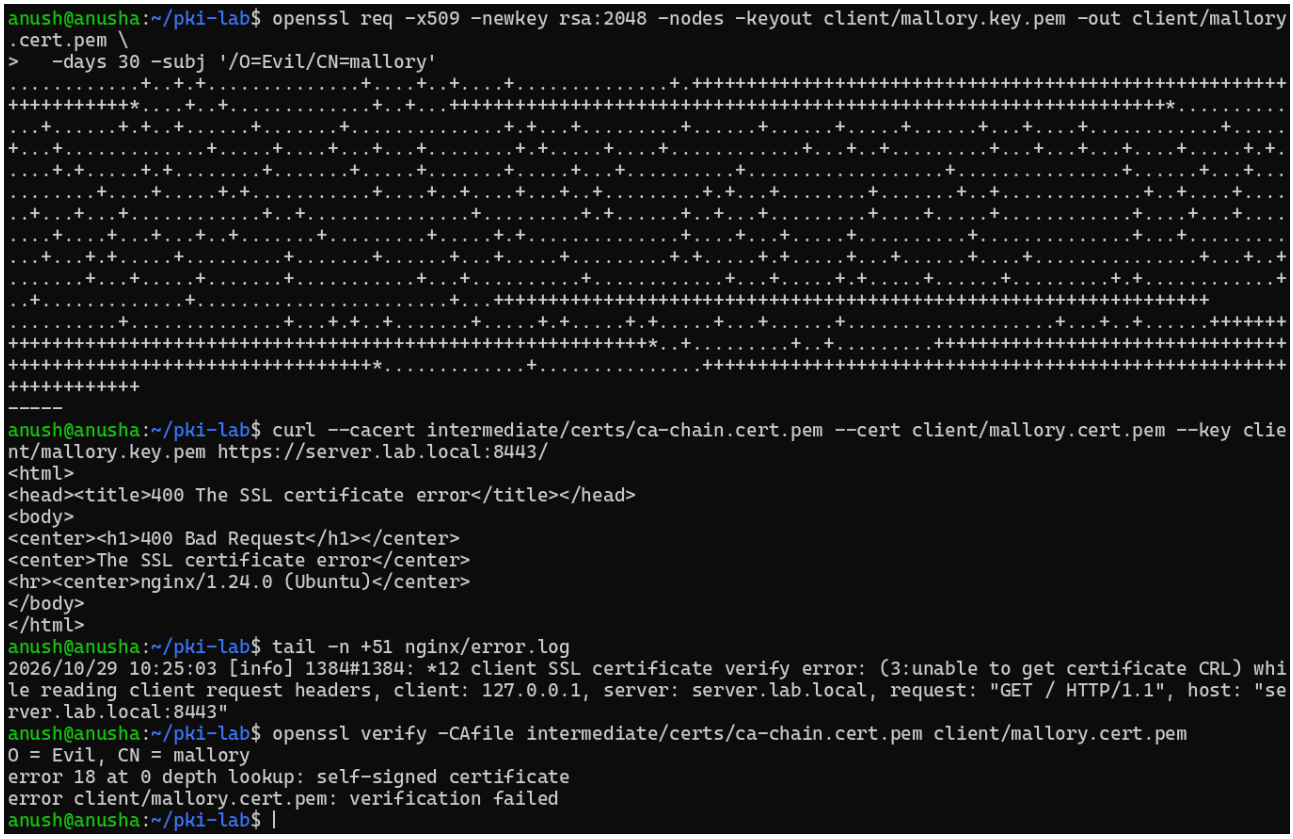


Figure 53: Part G.3: wrong purpose (server certificate used as a client certificate)



Figure 54: Part G.4: hostname mismatch (wrong.lab.local)

```
anush@anusha:~/pki-lab$ curl --resolve wrong.lab.local:8443:127.0.0.1 --cacert intermediate/certs/ca-chain.cert.pem
\
>   --cert client/alice.cert.pem --key client/alice.key.pem https://wrong.lab.local:8443/
curl: (60) SSL: no alternative certificate subject name matches target host name 'wrong.lab.local'
More details here: https://curl.se/docs/sslcerts.html

curl failed to verify the legitimacy of the server and therefore could not
establish a secure connection to it. To learn more about this situation and
how to fix it, please visit the web page mentioned above.
anush@anusha:~/pki-lab$ tail -n +53 nginx/error.log
2026/10/29 10:25:03 [info] 1384#1384: *14 client closed connection while waiting for request, client: 127.0.0.1, ser
ver: 0.0.0.0:8443
anush@anusha:~/pki-lab$ openssl s_client -connect 127.0.0.1:8443 -servername wrong.lab.local -verify_hostname wrong.
lab.local \
>   -CAfile intermediate/certs/ca-chain.cert.pem -cert client/alice.cert.pem -key client/alice.key.pem < /dev/null 2
>&1 | grep -E 'verify error|Verify return code|Verification'
verify error:num=62:hostname mismatch
Verification error: hostname mismatch
Verify return code: 62 (hostname mismatch)
anush@anusha:~/pki-lab$
```

Figure 55: Part G.5: missing intermediate (leaf only as ssl_certificate), then restored

```
anush@anusha:~/pki-lab$ sed -i 's#server/server.fullchain.pem;#server/server.cert.pem;#' nginx/nginx.conf
anush@anusha:~/pki-lab$ grep -n "ssl_certificate " nginx/nginx.conf
10:    ssl_certificate         /home/anush/pki-lab/server/server.cert.pem;
anush@anusha:~/pki-lab$ nginx -p $PKI/nginx -c $PKI/nginx/nginx.conf -s reload
2026/10/29 10:25:03 [notice] 1944#1944: signal process started
anush@anusha:~/pki-lab$ curl --cacert intermediate/certs/ca-chain.cert.pem --cert client/alice.cert.pem --key client
/alice.key.pem https://server.lab.local:8443/
Hello CN=alice,O=CSE Lab | verify=SUCCESS | proto=TLSv1.3
anush@anusha:~/pki-lab$ echo
anush@anusha:~/pki-lab$ curl --cacert root/certs/root.cert.pem --cert client/alice.cert.pem --key client/alice.key.p
em https://server.lab.local:8443/
curl: (60) SSL certificate problem: unable to get local issuer certificate
More details here: https://curl.se/docs/sslcerts.html

curl failed to verify the legitimacy of the server and therefore could not
establish a secure connection to it. To learn more about this situation and
how to fix it, please visit the web page mentioned above.
anush@anusha:~/pki-lab$ tail -n +67 nginx/error.log
2026/10/29 10:25:05 [info] 1945#1945: *17 SSL_do_handshake() failed (SSL: error:0A000418:SSL routines::tlsv1 alert u
nknown ca:SSL alert number 48) while SSL handshaking, client: 127.0.0.1, server: 0.0.0.0:8443
anush@anusha:~/pki-lab$ openssl s_client -connect server.lab.local:8443 -CAfile root/certs/root.cert.pem \
>   -cert client/alice.cert.pem -key client/alice.key.pem < /dev/null 2>&1 | grep -E 'depth|verify error|Verify retu
rn code|Certificate chain| s:| i:'
depth=0 O = CSE Lab, CN = server.lab.local
verify error:num=20:unable to get local issuer certificate
depth=0 O = CSE Lab, CN = server.lab.local
verify error:num=21:unable to verify the first certificate
depth=0 O = CSE Lab, CN = server.lab.local
Certificate chain
 0 s:O = CSE Lab, CN = server.lab.local
   i:O = CSE Lab, CN = CSE Lab Intermediate CA
Verify return code: 21 (unable to verify the first certificate)
anush@anusha:~/pki-lab$ sed -i 's#server/server.cert.pem;#server/server.fullchain.pem;#' nginx/nginx.conf
anush@anusha:~/pki-lab$ grep -n "ssl_certificate " nginx/nginx.conf
10:    ssl_certificate         /home/anush/pki-lab/server/server.fullchain.pem;
anush@anusha:~/pki-lab$ nginx -p $PKI/nginx -c $PKI/nginx/nginx.conf -s reload
2026/10/29 10:25:05 [notice] 2018#2018: signal process started
anush@anusha:~/pki-lab$ curl --cacert root/certs/root.cert.pem --cert client/alice.cert.pem --key client/alice.key.p
em https://server.lab.local:8443/
Hello CN=alice,O=CSE Lab | verify=SUCCESS | proto=TLSv1.3
anush@anusha:~/pki-lab$ echo
anush@anusha:~/pki-lab$
```

Figure 56: Part G.6: revoked certificate (Bob)

```
anush@anusha:~/pki-lab$ curl --cacert intermediate/certs/ca-chain.cert.pem --cert client/bob.cert.pem --key client/b
ob.key.pem https://server.lab.local:8443/
<html>
<head><title>400 The SSL certificate error</title></head>
<body>
<center><h1>400 Bad Request</h1></center>
<center>The SSL certificate error</center>
<hr><center>nginx/1.24.0 (Ubuntu)</center>
</body>
</html>
anush@anusha:~/pki-lab$ tail -n +81 nginx/error.log
2026/10/29 10:25:07 [info] 2019#2019: *20 client SSL certificate verify error: (23:certificate revoked) while readin
g client request headers, client: 127.0.0.1, server: server.lab.local, request: "GET / HTTP/1.1", host: "server.lab.
local:8443"
anush@anusha:~/pki-lab$ openssl verify -crl_check -CAfile intermediate/certs/ca-chain.cert.pem -CRLfile intermediate
/crl/combined.crl.pem client/bob.cert.pem
O = CSE Lab, CN = bob
error 23 at 0 depth lookup: certificate revoked
error client/bob.cert.pem: verification failed
anush@anusha:~/pki-lab$
```

Figure 57: Part G: all client-certificate errors in nginx error.log

```
anush@anusha:~/pki-lab$ grep -E 'verify error|no required SSL certificate' nginx/error.log
2026/10/29 10:24:42 [info] 1114#1114: *2 client sent no required SSL certificate while reading client request header
s, client: 127.0.0.1, server: server.lab.local, request: "GET / HTTP/1.1", host: "server.lab.local:8443"
2026/10/29 10:24:42 [info] 1114#1114: *3 client sent no required SSL certificate while reading client request header
s, client: 127.0.0.1, server: server.lab.local, request: "GET / HTTP/1.1", host: "server.lab.local:8443"
2026/10/29 10:24:44 [info] 1285#1285: *6 client SSL certificate verify error: (23:certificate revoked) while reading
 client request headers, client: 127.0.0.1, server: server.lab.local, request: "GET / HTTP/1.1", host: "server.lab.l
ocal:8443"
2026/10/29 10:24:46 [info] 1347#1347: *7 client SSL certificate verify error: (3:unable to get certificate CRL) whil
e reading client request headers, client: 127.0.0.1, server: server.lab.local, request: "GET / HTTP/1.1", host: "ser
ver.lab.local:8443"
2026/10/29 10:25:02 [info] 1384#1384: *11 client SSL certificate verify error: (10:certificate has expired) while re
ading client request headers, client: 127.0.0.1, server: server.lab.local, request: "GET / HTTP/1.1", host: "server.
lab.local:8443"
2026/10/29 10:25:03 [info] 1384#1384: *12 client SSL certificate verify error: (3:unable to get certificate CRL) whi
le reading client request headers, client: 127.0.0.1, server: server.lab.local, request: "GET / HTTP/1.1", host: "se
rver.lab.local:8443"
2026/10/29 10:25:03 [info] 1384#1384: *13 client SSL certificate verify error: (26:unsuitable certificate purpose) w
hile reading client request headers, client: 127.0.0.1, server: server.lab.local, request: "GET / HTTP/1.1", host: "
server.lab.local:8443"
2026/10/29 10:25:07 [info] 2019#2019: *20 client SSL certificate verify error: (23:certificate revoked) while readin
g client request headers, client: 127.0.0.1, server: server.lab.local, request: "GET / HTTP/1.1", host: "server.lab.
local:8443"
anush@anusha:~/pki-lab$ nginx -p $PKI/nginx -c $PKI/nginx/nginx.conf -s stop
2026/10/29 10:25:07 [notice] 2080#2080: signal process started
anush@anusha:~/pki-lab$
```

Observed results

#	Scenario	curl	nginx error.log	OpenSSL
1	Expired client (eve)	HTTP 400 The SSL certificate error	10: certificate has expired	verify: error 10
2	Untrusted self-signed client (mallory)	HTTP 400 The SSL certificate error	3: unable to get certificate CRL (ssl_crl on)	verify: error 18 self-signed certificate
3	Wrong purpose (server cert as client)	HTTP 400 The SSL certificate error	26: unsuitable certificate purpose	verify -purpose sslclient: error 26
4	Hostname mismatch (wrong.lab.local)	curl (60): no alternative certificate subject name matches	client closed connection (no verify error)	s_client -verify_hostname: 62 hostname mismatch
5	Missing intermediate (leaf only)	chain in --cacert: works; Root only: (60) unable to get local issuer certificate	SSL alert 48: unknown ca	s_client: 20, then 21
6	Revoked (bob)	HTTP 400 The SSL certificate error	23: certificate revoked	verify -crl_check: error 23

Answers: Task G — Failure scenarios (root causes)

1. **Expired client (eve):** the date is outside eve's validity (1–2 Jan 2024) → **10: certificate has expired**.

2. **Untrusted client (Mallory):** her self-signed certificate doesn't chain to our Root. With `ssl_crl` on, nginx reports **3: unable to get certificate CRL** (no CRL for an unknown issuer); `openssl verify` reports **18: self-signed certificate**.

3. **Wrong purpose:** the server certificate's EKU is serverAuth only, not clientAuth → **26: unsuitable certificate purpose**.

4. **Hostname mismatch:** wrong.lab.local is not in the server's SAN → curl (60) "no alternative certificate subject name matches"; s_client **62: hostname mismatch**. The client aborts, so nginx logs no verify error.

5. **Missing intermediate:** the server sends only its leaf. With the full chain in `--cacert` it still works; with only the Root, curl fails "unable to get local issuer certificate" (s_client **20/21**), because the client can't link the leaf to the Root.

6. **Revoked (Bob):** serial 1002 is on the Intermediate's CRL (keyCompromise) and nginx has `ssl_crl` on → **23: certificate revoked**.

Reflection

What is the single most common reason real-world TLS deployments fail certificate validation, and how would you prevent it? (150 to 250 words)

Reflection
[handwritten answers attached]

Submission files

The zip file contains this report and the following files from ~/pki-lab. No private keys (*.key.pem) are included.

File	Contents
captures/keys.log	TLS 1.3 session secrets (SSLKEYLOGFILE)
captures/tls12.pcapng	TLS 1.2 capture
captures/tls13.pcapng	TLS 1.3 capture
client/alice.cert.pem	certificate
client/bob.cert.pem	certificate
client/eve.cert.pem	certificate
client/mallory.cert.pem	certificate
intermediate/certs/ca-chain.cert.pem	certificate chain
intermediate/certs/intermediate.cert.pem	certificate
intermediate/crl/combined.crl.pem	CRL
intermediate/crl/intermediate.crl.pem	CRL
intermediate/index.txt	CA database
intermediate/openssl.cnf	CA configuration
nginx/access.log	nginx access log
nginx/error.log	nginx error log (level info)
nginx/error_log_excerpts.txt	client-certificate errors only
nginx/nginx.conf	nginx configuration
root/certs/root.cert.pem	certificate
root/crl/root.crl.pem	CRL
root/index.txt	CA database
root/openssl.cnf	CA configuration
server/server.cert.pem	certificate
server/server.fullchain.pem	certificate chain